From the Diagnostic to Week 1: the foundations guide

After this guide you can read code and queries the way the machine does, reason about a business number without a formula, hold a sensible conversation about how a language model works, and keep all of it in a GitHub portfolio an interviewer will open.

137 minute read · 36 figures · 26,168 words

How to read this guide

You sat the diagnostic and you have the emailed report open beside you. This guide is built for the four days between that report and the first hour of Week 1, and it does the four things the diagnostic could not: it places each idea inside the programme, it adds the history and the real cases, it compresses each area into one picture you can redraw, and it shows where each idea gets tested.

Reading time is stated once at the top of each chapter, and it assumes you read at a working pace and stop to redraw each picture. The mini projects at the end of each chapter are the point of the guide; the reading exists to make them possible.

CHAPTER	WHAT IT ASSERTS	DIAGNOSTIC QUESTIONS IT REVISITS	READING	HANDS-ON
1 Python	Python reads data the way it is written, never the way you meant it	Q1 to Q12	13 min	90 min
2 SQL	A query runs in an order that is not the order you write	Q13 to Q20	11 min	90 min
3 Numbers	Most wrong numbers are right arithmetic on the wrong denominator	Q21 to Q28	12 min	60 min
4 Language models	A language model is a sampler, and a prompt is a specification	Q28, Q33, Q34	11 min	60 min
5 Business problems and judgment	An unfamiliar problem is decomposed, never solved in one move	Q29 to Q40	9 min	60 min
6 pandas	A DataFrame is the SQL pipeline with a different spelling	none, new	7 min	90 min
7 Excel	A PivotTable is GROUP BY for people who will never see your code	none, new	5 min	45 min
8 GitHub	Your GitHub is the portfolio interviewers actually open	none, new	13 min	60 min
9 The diagnostic, worked	Every question, its answer, the steps, and why the other options fail	Q1 to Q40	44 min, as a reference	
The path	Videos, reading and the six mini projects, in order		5 min	

Every historical claim, organisation and number in this guide was checked against a live source on 29 September 2026, and the source is named where the claim appears. Anything marked as this course's own construction is a teaching device, not an industry standard.

The forty questions, in sequence

This map is the spine of the guide. Chapters 1 to 8 teach the ideas in the order that builds them, which is not the order of the paper; this table restores the paper's order, and every row links to the worked item in Chapter 9.

Q	SECTION	WHAT IT ASKED	ANSWER	TAUGHT IN	WORKED AT
Q1	A	Predict the output: types do not convert on their own	C	Chapter 1	Q1 in Chapter 9
Q2	A	Spot the bug, if there is one: two names, one list	B	Chapter 1	Q2 in Chapter 9
Q3	A	Trace the flow (pseudocode): accumulator and running best, strict comparison on a tie	D	Chapter 1	Q3 in Chapter 9
Q4	A	Predict the output: counting labels with dict.get	A	Chapter 1	Q4 in Chapter 9
Q5	A	Fix the code: reading a nested JSON response	B	Chapter 1	Q5 in Chapter 9
Q6	A	Predict the output: truthiness drops zero as well as None	D	Chapter 1	Q6 in Chapter 9
Q7	A	Which line: integer division hides the fraction	A	Chapter 1	Q7 in Chapter 9
Q8	A	Predict the output: a default argument is created once	C	Chapter 1	Q8 in Chapter 9
Q9	A	Fix the code: defensive conversion: skip and count the bad rows	A	Chapter 1	Q9 in Chapter 9
Q10	A	Spot the bug: sorted() on tuples orders by the first element	D	Chapter 1	Q10 in Chapter 9
Q11	A	Predict the output: braces inside a format template are placeholders	B	Chapter 1	Q11 in Chapter 9
Q12	A	Fix the code: a repeated inner list is one object	C	Chapter 1	Q12 in Chapter 9
Q13	B	Predict the result: cOUNT(*) against COUNT(column) with NULL	B	Chapter 2	Q13 in Chapter 9
Q14	B	Fix the query: integer division in SQL	A	Chapter 2	Q14 in Chapter 9
Q15	B	Predict the result: wHERE before grouping, HAVING after	D	Chapter 2	Q15 in Chapter 9
Q16	B	Spot the bug: nULL in a comparison	C	Chapter 2	Q16 in Chapter 9
Q17	B	Spot the bug and fix it: a WHERE on the right table turns a LEFT JOIN into an inner join	A	Chapter 2	Q17 in Chapter 9
Q18	B	Predict the result: aggregates cannot sit in WHERE	B	Chapter 2	Q18 in Chapter 9
Q19	B	Predict the result: latest row per group with ROW_NUMBER	D	Chapter 2	Q19 in Chapter 9

Q	SECTION	WHAT IT ASKED	ANSWER	TAUGHT IN	WORKED AT
Q20	B	Fix the query: join fan-out inflates a SUM	C	Chapter 2	Q20 in Chapter 9
Q21	C	Reason with numbers: ratios of percentage changes	D	Chapter 3	Q21 in Chapter 9
Q22	C	Reason with numbers: median against mean under one extreme value	A	Chapter 3	Q22 in Chapter 9
Q23	C	Reason with numbers: compare rates, not counts	B	Chapter 3	Q23 in Chapter 9
Q24	C	Two statements: percentage changes do not add	C	Chapter 3	Q24 in Chapter 9
Q25	C	Reason with numbers: small counts move for no reason	A	Chapter 3	Q25 in Chapter 9
Q26	C	Reason with numbers: base rates: precision from recall and prevalence	D	Chapter 3	Q26 in Chapter 9
Q27	C	Reason with numbers: mix shift can reverse a total	B	Chapter 3	Q27 in Chapter 9
Q28	C	Estimate: token economics, multi-step estimate	C	Chapter 3 and 4	Q28 in Chapter 9
Q29	D	Case: Kalpa Retail, Q1 against Q2: which branch moved	A	Chapter 5	Q29 in Chapter 9
Q30	D	Case: Kalpa Retail, Q1 against Q2: test a hypothesis against the table	C	Chapter 5	Q30 in Chapter 9
Q31	D	Case: Kalpa Retail, Q1 against Q2: the fair comparison	B	Chapter 5	Q31 in Chapter 9
Q32	D	Case: Kalpa Retail, Q1 against Q2: like-with-like windows before presenting	D	Chapter 5	Q32 in Chapter 9
Q33	D	Scenario: LLM intuition: sampling variance and an under-specified prompt	A	Chapter 4	Q33 in Chapter 9
Q34	D	Scenario: LLM intuition: a small hand-labelled set is the fastest trust check	B	Chapter 4	Q34 in Chapter 9
Q35	E	Best and worst: disclosing an error you found yourself	best C, worst A	Chapter 5	Q35 in Chapter 9
Q36	E	Best and worst: owning what you do not yet understand	best B, worst D	Chapter 5	Q36 in Chapter 9
Q37	E	Best and worst: taking criticism in front of others	best D, worst B	Chapter 5	Q37 in Chapter 9
Q38	E	Best and worst: deadline against a known defect	best A, worst C	Chapter 5	Q38 in Chapter 9

Q	SECTION	WHAT IT ASKED	ANSWER	TAUGHT IN	WORKED AT
Q39	E	Best and worst: asking for help at the right time	best C, worst D	Chapter 5	Q39 in Chapter 9
Q40	E	Best and worst: holding a finding under pushback	best B, worst A	Chapter 5	Q40 in Chapter 9

Where Week 0 sits, and the one question that runs through every chapter

The programme has ten modules across twenty weeks. Week 0 is the ground they stand on: Python and SQL are used in class from the first hour of Week 1, Build 1 in Week 3 turns data into an insight and presents it, and the interviews that start around Week 15 test exactly the habits this guide names. The terrain figure below is the map every later set of notes will mark; today, every cell in the bottom band is being filled.

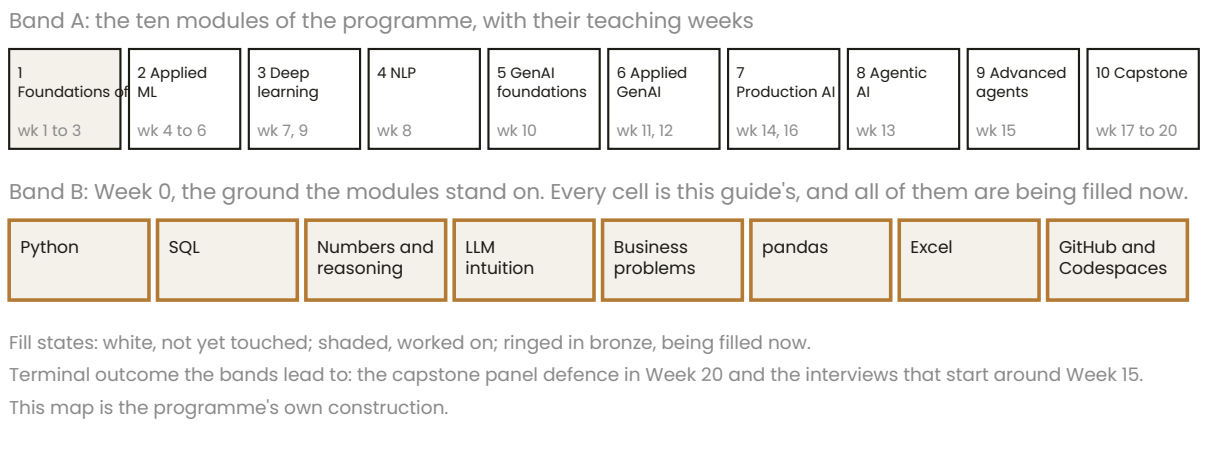


Figure 0. The ten modules with their teaching weeks in the top band, and the eight Week 0 areas in the bottom band. Every bottom cell is ringed because every one is being filled this week.

The running thread is Meera Raghavan's question from the diagnostic case. She is the CEO of Kalpa Retail, a fictional company you will meet again from Week 1, and her question is the one every chapter applies its idea to: revenue fell from Q1 to Q2, where did it go?

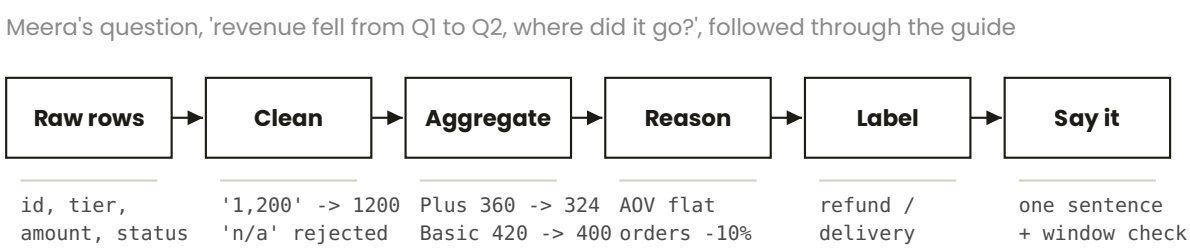


Figure 1. Meera's question followed from raw order rows to the sentence she hears. Each chapter shows what happens to this same question when its idea is applied.

Chapter 1. Python reads data the way it is written, never the way you meant it

Every Python bug in the diagnostic was the same bug wearing different clothes: the code did exactly what was written, and the writer expected something else. Reading time: 13 minutes.

Diagnostic questions this chapter revisits, in the paper's order: Q1, Q2, Q3, Q4, Q5, Q6, Q7, Q8, Q9, Q10, Q11, Q12; each is worked step by step in Chapter 9.

What you can now do

You can predict what a short piece of Python prints before running it. You can explain why two names can change together and how to stop it. You can trace a loop by hand with two variables and a running total. You can read a nested API response and write the one path that reaches the text inside it. You can turn a crash on a bad value into a counted, logged rejection. You can read a traceback from the bottom line up and say which line to look at.

Where this sits

What this chapter covers. Values and their types, names and objects, loops and accumulators, functions and their defaults, dicts and lists as the shape of JSON, and errors as information. Each was worked in the diagnostic through one question; here each gets the mechanism behind the question. Files, modules and notebooks are mentioned at the end and worked in Week 1.

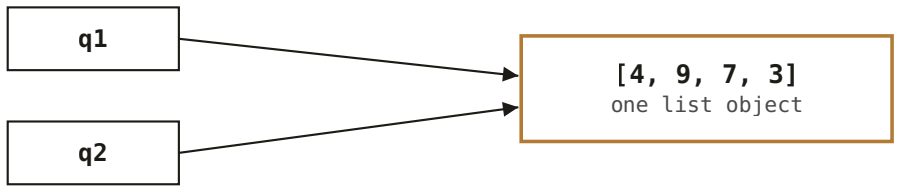
Placement. Python is the first cell of the bottom band and the language of every notebook from Week 1 onward. Module 1, Foundations of AI and Data, assumes it from its first session.

Outcome tie. The specific moment this chapter serves is the first afternoon of Week 1, when a notebook on Kalpa's order rows fails on a value like `'1,200'` and the room has to say why before anyone touches the code.

What was left out. Classes and objects you define yourself arrive in Week 2, and pandas, which replaces most hand-written loops over rows, has its own chapter here.

The picture to remember: the name-and-object board

Names on the left point at objects on the right. Assignment moves an arrow; it never copies an object.



q2 = q1 adds the second arrow. q2.append(3) changes the object, so q1 sees 4 elements.
q2 = q1.copy() would build a second object and point q2 at that instead.

Figure 2. Two names, one object. Assignment moves an arrow; only a method or a copy touches the object. Call this the board; every section below comes back to it.

> **ORIGIN** Python was released by Guido van Rossum in February 1991 as version 0.9.0, written at the CWI research institute in Amsterdam as a successor to a teaching language called ABC (sources: the Python 0.9.1 archive on GitHub; luisllamas.es, history of Python). The board above is why its assignment statement has behaved the same way for thirty-five years.

Values have types, and Python never guesses

Q1 added an integer 0 to the string `"120"` and Python refused with `TypeError`. That refusal is the design. A value carries its type with it, and the operators ask the type what to do: `+` on two strings joins them, `+` on two numbers adds them, `+` across the two has no meaning and stops the program rather than inventing one.

Value as written	Type	What + does	What int() or float() does
"120"	str	joins strings	reads the digits: 120
120	int	adds numbers	unchanged
120.0	float	adds numbers	int() truncates to 120
None	NoneType	raises TypeError	raises TypeError
True	bool	counts as 1	1

Figure 3. The five types the diagnostic used, what `+` does to each, and what conversion does. Conversion is always something you ask for.

Applied to Meera's question: the order rows arrive from a CSV, and every field in a CSV is text. Revenue by tier is impossible until each amount has been converted with `float(amount.replace(",", ""))`, and the only honest place for that conversion is one function, called on every row, that either returns a number or raises.

> **WATCH OUT** Truthiness drops zeros. `[s for s in sales if s]` removes `None` and also removes `0`, because in an `if` test both are false (Q6). A report that filters "empty" values this way silently deletes every genuine zero. The tell is a count of rows that is smaller than the count of non-missing values.
> **CALLBACK** Diagnostic Q1, Q6 and Q14 were the same lesson: the machine keeps types strictly, and SQL's integer division in Q14 is the same strictness in a different language.

One object, many names

Q2 surprised most of the room. `q2 = q1` did not copy the list; it added a second name pointing at the same object, so `q2.append(3)` showed through `q1`. On the board, assignment only moves an arrow. The rule is short: assignment never copies, methods that end in place change the object, and `copy()` or `list()` makes a new object.

Q12 was the same rule one level down. `[[0] * 3] * 2` builds one inner list and repeats the arrow to it twice, so writing through row 0 shows in row 1. `list()` and `copy()` copy only the outer list, which is why they fail as fixes; only a comprehension, `[[0] * 3 for _ in range(2)]`, builds a fresh inner list each time.

> **IN THE FIELD** Ned Batchelder's PyCon 2015 talk, "Facts and Myths about Python names and values", was built around exactly this board and is still the reference the Python community links when the question comes up; the written version on nedbatchelder.com carries the same figures (source: us.pycon.org 2015 schedule; nedbatchelder.com/text/names.html).

Applied to the thread: when you build a totals dictionary per tier and then "copy" it to try an adjustment, you are adjusting the original unless you copied. The check is `a is b`, which is `True` when two names share one object.

Loops you can trace by hand

Q3 was pseudocode, and the diagnostic put it there to see whether you can hold two variables in your head for four rows. The method is a table, not talent: one column per variable, one row per iteration, and the comparison written out where it matters.

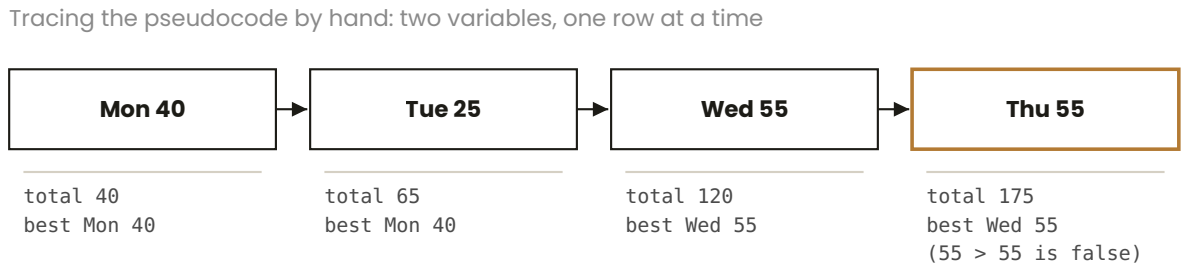


Figure 4. The trace table for Q3. The strict `>` on the tie is the whole item: Thursday's 55 does not replace Wednesday's 55.

Q4 was the same habit on a dictionary. `count.get(lab, 0) + 1` reads "the current count, or zero if this label is new, plus one", and by the end of the loop the dictionary holds one key per

distinct label. Counting by key with `get` is the pattern that turns a list of LLM labels into a table you can reason about, and you will write it in Week 1 before pandas takes it over.

Applied to the thread: revenue by tier is this loop with amounts instead of ones. `totals[tier] = totals.get(tier, 0) + amount`, once per row, gives Plus 360 and Basic 420 for Q1 without a single library.

Functions, defaults and what a call returns

Q8 was the interview classic. `def tag_order(order_id, tags=[])` evaluates the empty list once, when `def` runs, and every call that omits `tags` reuses that one object, so the second call sees the first call's tag.

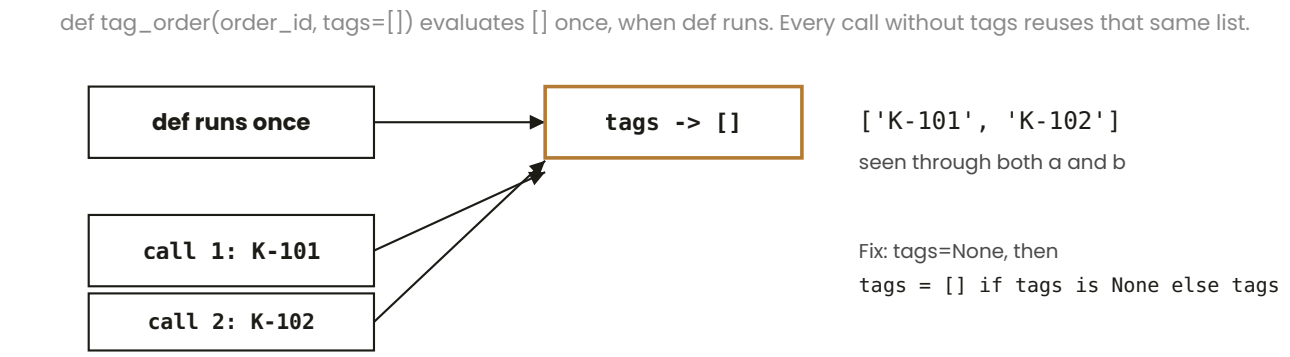


Figure 5. One default object, shared by every call. The idiom `tags=None` followed by `tags = [] if tags is None else tags` builds a fresh list per call.

Q7 was about what a function returns rather than what it prints. `totals[tier] // counts[tier]` is floor division and hands back a whole number; the calculator showed 833.33 because it used true division. When a function's answer differs from a calculator's, look at the operator before the data.

> **WATCH OUT** Methods that work in place return `None`. `ranked = scores.sort()` sorts `scores` and sets `ranked` to `None`; `sorted(scores)` returns the new list. Half the "my variable is None" questions on any forum are this.

Dicts, lists and JSON: the shape of data

Every API you call this programme returns JSON, and JSON is only two shapes nested: braces are dictionaries you index by key, brackets are lists you index by position. Q5's response put a list inside `"choices"`, so the path had to say `[0]` before it could say `["message"]`.

The shape of an API response: braces are dicts (key lookup), brackets are lists (integer index).

```
{"choices": [
{"message": {"role": "assistant", "content": "Refund approved"}}
],
"usage": {"prompt_tokens": 412, "completion_tokens": 9}}
```

one path, read left to right:

```
resp["choices"][0]["message"]["content"]
```

Figure 6. Reading a response is reading its brackets. One path, left to right, one index per level.

Q10 belongs here too. `sorted()` orders tuples element by element from the first, so a list of `(name, spend)` pairs sorts by name; `key=lambda c: c[1]` says which element to sort on. Q11 is the JSON trap in reverse: `str.format()` treats every `{...}` as a placeholder, so a JSON example inside a template raises `KeyError`, and the fix is doubled braces or building the JSON part separately.

Applied to the thread: the ticket classifier in Chapter 4 returns a response like Q5's, and the two lines that read `content` out of it are the two lines you will copy into every notebook that calls a model.

Errors are information, not failure

Q9 asked for a rewrite that skipped every bad value and counted it, whatever the value looked like. The comprehension crashed on `'n/a'`; skipping a fixed list of bad strings misses the next one; a `try` around the whole loop drops every row after the first failure. The only shape that survives real data is a `try` inside the loop.



One try/except inside the loop.
A try around the whole loop stops at the first bad value and drops every row after it.

Figure 7. One `try` per value. The bad rows are kept in a list, counted, and reported, which is what Anand asked for.

A traceback reads from the bottom. The last line names the exception and the reason (`TypeError: unsupported operand type(s) for +: 'int' and 'str'`); the line above it points at the code that raised it. Read those two lines before anything else, and you will fix most errors without searching.

> **IN THE FIELD** The official Python tutorial's chapter on errors makes the same point with the same example, `'2' + 2`, and shows the current interpreter marking the exact span of code with `~~~~^~~` under the offending expression (source: docs.python.org, The Python Tutorial, section 8, Errors and Exceptions).

Files, modules and the notebook, briefly

Three things the diagnostic did not test and Week 1 uses on Monday. A file is opened with `with open("orders.csv") as f:` and closed for you at the end of the block; `import csv` gives a reader that splits each line on commas. An `import` line loads a module, which is just another Python file, once. A notebook runs cells in the order you run them, not the order they appear, so a restart followed by "run all" is the only proof a notebook works.

Where this shows up in the work

The first Monday. A colleague's script totals revenue as `0` because every amount was a string and a bare `except` swallowed the errors. Ten minutes with the traceback finds it; a day of guessing does not.

The code review. A function with `results=[]` as a default passes every test that calls it once and fails in production on the second request. Reviewers who know the board catch it in the signature.

The model call. A notebook that reads `resp["choices"]["message"]` crashes on the first live call after working on a hand-typed dict. The cost is a wasted API call, plus the meeting where it was meant to run.

Try this yourself

No-code self-check. Answer in your head, then check against the key. (1) After `a = [1, 2]; b = a; b += [3]`, what does `len(a)` return? (2) Which returns a new list: `sorted(x)` or `x.sort()`? (3) In `def f(n, seen=set())`, how many set objects exist after three calls that omit `seen`? Key: (1) 3, since `+=` on a list works in place; (2) `sorted(x)`; (3) one. A miss on (1) or (3) sends you back to the board; a miss on (2) to the functions section.

Mini project 1, Python: clean and total Kalpa's orders. Create a repository named `w00-diagnostic-python` (Chapter 8 shows how) and a notebook with three cells. Cell one holds a list of ten order rows as tuples of `(order_id, tier, amount_text, status)`, with at least two bad amounts such as `'n/a'` and `''` and one cancelled order. Cell two defines `to_amount(text)` and a loop that builds `totals` by tier for paid orders only, collecting rejected rows in a list. Cell three prints the totals, the rejected rows, and an `assert` that the sum of the totals equals the sum of the clean amounts. Self-check: the notebook restarts and runs top to bottom with no error; the rejected list has exactly the bad rows you planted; changing one amount to `'0'` keeps the row (a truthiness test would drop it).

Where this gets tested

Interview question. "What does `b = a` do when `a` is a list?" Tested: whether you hold the board. Strong answer: it binds a second name to the same object, so mutation through either

name is visible through both; a copy needs `a.copy()`, `list(a)` or a slice. Weak answer: "it copies the list", or a correct answer with no mention of mutation.

Interview question. "Why is a mutable default argument a bug?" Tested: whether you know when defaults are evaluated. Strong answer: once, at definition time, so the object is shared across calls; the idiom is `None` plus a fresh object inside. Weak answer: "it is bad practice", with no mechanism.

Interview question. "Your script crashes on one bad row in a million. What do you change?" Tested: error handling as a design choice. Strong answer: a `try` per row that records the rejects with their reason and continues, plus a count reported at the end. Weak answer: a `try` around the loop, or `except: pass`.

Interview question. "Read this traceback and tell me the line to look at." Tested: whether you read from the bottom. Strong answer names the exception, the message, and the nearest line in your own code, in that order.

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
Object	A value living in memory, with a type	The board	The list `[4, 9, 7]`
Name	A label bound to an object by assignment	The board	`q1`, `q2`
Mutable	Can be changed in place	Names section	Lists and dicts; strings and tuples are not
Truthiness	How a value behaves in an `if` test	Types section	`0`, `None`, `""` and `[]` are false
Floor division	`//`, which discards the fraction	Functions section	`2500 // 3` is `833`
Traceback	The report Python prints when an exception is not caught	Errors section	Read the last two lines first
Comprehension	A loop that builds a list in one expression	Names section	`[[0] * 3 for _ in range(2)]`

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	Ned Batchelder, "Facts and Myths about Python names and values", PyCon 2015, youtube.com/watch?v=_AEJHKGk9ns	30 min	The board, animated, by the person who drew it
2	The same talk in text, nedbatchelder.com/text/names.html	20 min	For re-reading after the mini project
3	The Python Tutorial, section 8, Errors and Exceptions, docs.python.org/3/tutorial/errors.html	25 min	The tracebacks you will actually see, explained by the source
4	Mini project 1	90 min	The loop, the conversion and the rejects, in a notebook you keep

Chapter 2. SQL runs in an order that is not the order you write

A query is read by the database from `FROM` outward, not from `SELECT` down, and every SQL question in the diagnostic was decided by that order or by the meaning of NULL. Reading time: 11 minutes.

Diagnostic questions this chapter revisits, in the paper's order: Q13, Q14, Q15, Q16, Q17, Q18, Q19, Q20; each is worked step by step in Chapter 9.

What you can now do

You can say which rows a query returns before running it. You can explain why NULL neither equals nor differs from anything and what that does to a filter. You can place a condition in `WHERE`, `HAVING` or `ON` on purpose. You can predict when a join multiplies rows and stop it. You can keep every row while adding a per-group number with a window function. You can write the query that answers Meera's question in one statement.

Where this sits

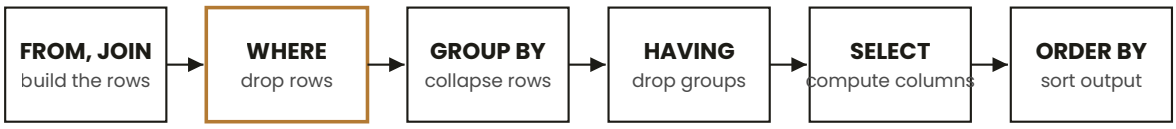
What this chapter covers. Rows and NULL, the execution pipeline, types and integer division, what a LEFT JOIN keeps, join fan-out, and window functions. Common table expressions, subqueries and dates are mentioned at the end and worked in Week 2.

Placement. SQL is the second cell of the bottom band. Module 1 runs it against a live PostgreSQL database from Week 2, and Build 1 in Week 3 is written in it.

Outcome tie. The specific moment is Build 1 in Week 3, when a group's revenue-by-category total comes out higher than the raw table's total and the group has to find the join that did it before the presentation.

What was left out. Indexes, query plans and writing to the database arrive in Week 2; the modelling of tables, which is a design skill, arrives with the first data-to-insight build.

The picture to remember: the pipeline



Written order: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY. Executed order: this row.
An aggregate such as SUM exists only from GROUP BY onward, which is why WHERE cannot see it.

Figure 8. The order the database executes a query in. Every SQL question in the diagnostic was decided by where a condition sat in this row. Call this the pipeline.

> **ORIGIN** E. F. Codd published "A Relational Model of Data for Large Shared Data Banks" in June 1970 at IBM's San Jose laboratory; Donald Chamberlin and Raymond Boyce turned his algebra into SEQUEL, later SQL, in 1974, and ANSI accepted SQL as a standard in 1986 (sources: Britannica, Edgar Frank Codd; historyofinformation.com, Chamberlin and Boyce 1974; historyofdatascience.com, key dates).

Rows and the meaning of NULL

Q13 and Q16 were both about one row: the order whose amount was NULL. NULL is not a value; it is the absence of one, and every comparison with it is unknown rather than true or false. `WHERE amount <> 800` keeps rows whose test is true, so the NULL row is dropped alongside the 800 row, and Kavya gets 3 where she expected 4.

Three-valued logic: a comparison with NULL is unknown, and WHERE keeps only true

amount	amount <> 800	WHERE keeps it?	COUNT(*)	COUNT(amount)	SUM(amount)
1200	true	yes	1	1	1200
NULL	unknown	no	1	0	ignored
800	false	no	1	1	800
300	true	yes	1	1	300

Figure 9. Three-valued logic on four rows. `COUNT(*)` counts rows, `COUNT(amount)` counts values, and `SUM` ignores NULL rather than becoming NULL.

The three aggregates in Q13 fall out of the same table: after `WHERE status <> 'cancelled'` four rows remain, three of them with an amount, summing to 2000. When you want NULL rows kept, say so: `amount <> 800 OR amount IS NULL`.

> **IN THE FIELD** Codd himself extended the relational model in 1979 to handle missing information, in "Extending the Database Relational Model to Capture More Meaning" (ACM Transactions on Database Systems, 1979); the three-valued logic you met in Q16 is a direct descendant of that paper (source: the VLDB bibliography of Codd's papers).

The pipeline: WHERE, GROUP BY, HAVING

Q15 and Q18 were the pipeline read twice. `WHERE` runs before grouping and sees single rows; `HAVING` runs after grouping and sees group totals. That is why `WHERE SUM(amount) > 1000` fails outright on PostgreSQL, and why `WHERE status = 'paid'` removed the cancelled Plus order before `HAVING COUNT(*) > 1` kept only Basic.

Applied to the thread, the query for revenue by tier is the pipeline written out once:

```
SELECT tier, COUNT(*) AS orders, SUM(amount) AS revenue
FROM orders
WHERE status = 'paid' AND order_date >= '2026-04-01' AND order_date < '2026-07-01'
GROUP BY tier
ORDER BY revenue DESC;
```

The date range is written as a half-open interval, `>=` the first day and `<` the day after the last, which is the one form that never miscounts the last day.

> **CALLBACK** Chapter 1's loop, `totals[tier] = totals.get(tier, 0) + amount`, is this `GROUP BY` written by hand. Every `GROUP BY` you write from now on is that loop delegated to the database.

Types, and integer division

Q14 returned 0 for a cancel rate of one in five because both operands were integers and PostgreSQL divides integers as integers. Multiplying by `1.0` or casting one side makes the division decimal. The same strictness lives in the types of every column: a date stored as text sorts as text, and `'2026-1-5'` lands before `'2026-01-20'`.

> **WATCH OUT** `COUNT(status = 'cancelled')` looks like it counts the cancelled rows and instead counts every row where the expression is not NULL, which is all of them. Count a condition with `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` or, on PostgreSQL, `COUNT(*) FILTER (WHERE status = 'cancelled')`.

What a LEFT JOIN keeps

Q17 was the trap that catches working analysts. A `LEFT JOIN` keeps every customer, filling the order columns with NULL for those who never ordered; a `WHERE` on an order column then drops exactly those NULL rows, and the join behaves like an inner join.

LEFT JOIN keeps every customer; a WHERE on the right-hand table then throws the NULL rows away.

customers	orders after LEFT JOIN	WHERE order_date >= '2026-01-01'
C1	1, 2026-01-15	kept
C1	3, 2026-03-02	kept
C2	2, 2025-11-03	dropped by WHERE
C3	4, 2026-02-20	kept
C4	NULL, NULL	dropped by WHERE

Move the date test into ON and it filters the orders side before the join, so C2 and C4 survive with a count of 0.

Figure 10. The join keeps C2 and C4; the `WHERE` throws them out. A condition in `ON` filters the right side before the join and every customer survives with a count of zero.

The rule to say out loud: a condition on the right-hand table goes in `ON` when you want to keep the left-hand rows, and in `WHERE` when you want to drop them.

Fan-out, the silent multiplier

Q20 inflated LLM spend by joining a table with several rows per ticket. Each call row was repeated once per tag, and `SUM` added the repeats. Nothing errors, the numbers look plausible, and the total disagrees with the raw table by the exact amount of the repeated rows.

A one-to-many join repeats every left-hand row once per match. SUM then counts the repeats.



Drop the join the question does not need, or aggregate the many side first and join the totals.
SUM(DISTINCT) is not a fix: it also merges two genuinely equal costs into one.

Figure 11. One call row, two tags, two copies. The cheap check is the one from Q20: compare the joined total with the raw table's total before you trust either.

The fix is structural: drop the join the question does not need, or aggregate the many side to one row per key first and join the totals. `SUM(DISTINCT)` is not a fix, because it also merges two identical costs.

> **WATCH OUT** The tell for fan-out is a row count that grew when you added a join that was supposed to add columns. Print `COUNT(*)` before and after every join in a build week and you will never present an inflated total.

Window functions: every row keeps its seat

Q19 needed the latest call per ticket. `GROUP BY` would have collapsed the calls to one row per ticket and lost the call id; `ROW_NUMBER() OVER (PARTITION BY ticket_id ORDER BY call_id DESC)` numbers each ticket's calls without collapsing anything, and `WHERE rn = 1` in an outer query keeps the latest.

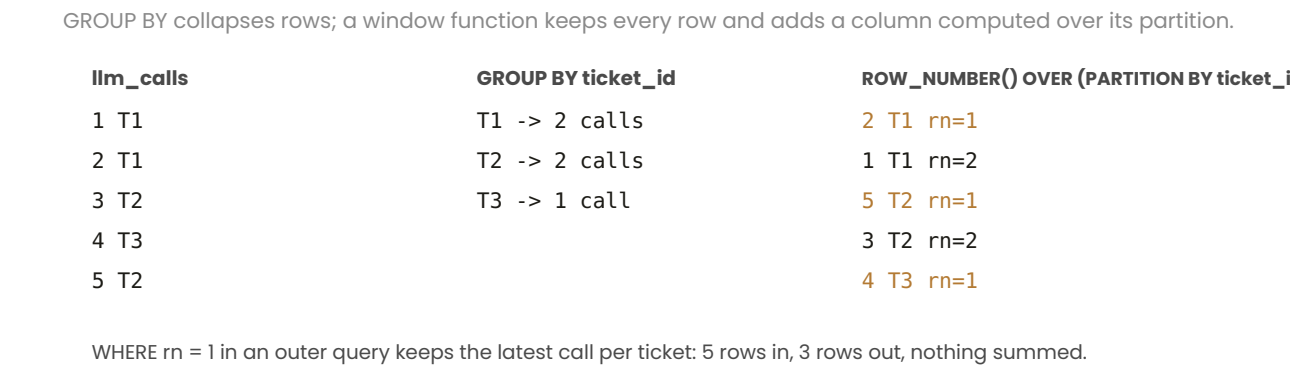


Figure 12. `GROUP BY` collapses; a window function annotates. The filter on `rn` has to live in an outer query because a window value does not exist yet when `WHERE` runs.

> **IN THE FIELD** Oracle shipped analytic functions with an `OVER` clause in Oracle 8i in 1998, five years before the ISO standard added window functions in SQL:2003; PostgreSQL, the database this programme runs on, supports them in every currently maintained version, 14 through 18 as of the February 2026 releases (sources: learnsql.com, history of window functions; mariadb.com, window functions overview; postgresql.org release notes, 26 February 2026).

Subqueries, common table expressions and dates, briefly

A subquery is a query used as a table; a common table expression, written `WITH totals AS (...)`, is the same thing with a name, and the name is what makes a long query readable. Dates on PostgreSQL are a real type with arithmetic: `order_date + INTERVAL '90 days'` and `DATE_TRUNC('month', order_date)` both work, and both fail on text. In Week 2 you connect VS Code to the programme's PostgreSQL instance through Codespaces and run `.sql` files against it; nothing installs on your laptop.

Where this shows up in the work

The build week presentation. A category total that exceeds the raw total by 40 percent is a fan-out, and a panel member will ask for the row count before and after the join. Having it on the slide is the difference between a question and a correction.

The first Finance request. "Customers with no orders this quarter" is a `LEFT JOIN` with the date test in `ON` and `WHERE o.order_id IS NULL`. Written with the test in `WHERE`, the query returns nobody and the report says the business has no dormant customers.

The dashboard that lies. A cancel rate that reads 0 percent for a year is integer division. It costs nothing to write `* 1.0`, and it costs a quarter of wrong decisions not to.

Try this yourself

No-code self-check. (1) A `LEFT JOIN` from customers to orders followed by `WHERE orders.amount > 0` returns which customers? (2) After `GROUP BY tier`, can you `SELECT order_id`? (3) `SELECT 7 / 2` on PostgreSQL returns what? Key: (1) only customers with a qualifying order, since the NULL rows fail the test; (2) no, unless it is aggregated, because one output row stands for many; (3) `3`. A miss on (1) sends you to the join section, on (2) to the pipeline, on (3) to types.

Mini project 2, SQL: Meera's question in one statement. In your `w00-diagnostic-sql` repository, create `orders.sql` that builds a small `orders` table with twelve rows across three tiers and two quarters, including one NULL amount and two cancelled orders, and a `customers` table with one customer who never ordered. Then write four queries in `answers.sql`: revenue and order count by tier and quarter for paid orders; the average order value by tier as a decimal; every customer with their count of Q2 orders, including zero; and the latest order per customer using `ROW_NUMBER`. Run them in a Codespace against PostgreSQL, or in any SQL playground, and paste the four result tables into the README. Self-check: the tier totals add up to the raw table's paid total; the customer with no orders appears with 0; the average order value is not a whole number; the `ROW_NUMBER` query returns exactly one row per customer.

Where this gets tested

Interview question. "What is the difference between `WHERE` and `HAVING`?" Tested: the pipeline. Strong answer: `WHERE` filters rows before grouping and cannot see aggregates; `HAVING` filters groups after, and can. Weak answer: "`HAVING` is for aggregates", with no mention of order.

Interview question. "Why did my `LEFT JOIN` return the same rows as an inner join?" Tested: NULL and `WHERE`. Strong answer: a `WHERE` on the right-hand table discards the NULL-filled rows; move the condition into `ON` or add `OR col IS NULL`. Weak answer: "the join type was wrong".

Interview question. "Your total is bigger than the source table's total. Name the cause and the check." Tested: fan-out. Strong answer: a one-to-many join repeated rows; compare row counts before and after each join, and aggregate the many side first. Weak answer: "use `DISTINCT`".

Interview question. "When would you use a window function instead of `GROUP BY`?" Tested: whether you know the difference in output shape. Strong answer: when you need a per-group number on every row, such as rank, running total or latest per key, without collapsing rows.

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
NULL	The absence of a value; comparisons with it are unknown	Rows section	`amount <> 800` drops the NULL row
Three-valued logic	True, false and unknown as the results of a comparison	Rows section	`WHERE` keeps only true
Aggregate	A function that turns many rows into one value	Pipeline section	`SUM`, `COUNT`, `AVG`
Fan-out	Row multiplication caused by a one-to-many join	Fan-out section	Two tags per ticket doubled the spend
Window function	A per-row calculation over a partition, without collapsing	Windows section	`ROW_NUMBER() OVER (...)`
Half-open interval	A date range with `>=` start and `<` next start	Pipeline section	`>= '2026-04-01' AND < '2026-07-01'`

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	PostgreSQL documentation, Tutorial, chapter 2, The SQL Language, postgresql.org/docs/current/tutorial-sql.html	40 min	The database you will use, explained by its own project, with joins and aggregates
2	freeCodeCamp, SQL Tutorial, Full Database Course for Beginners, youtube.com/watch?v=HXV3zeQKqGY	4 h, watch the joins and nested queries chapters first	Long, MySQL-based, and the clearest walk from tables to joins on YouTube; every idea transfers to PostgreSQL
3	Mini project 2	90 min	The four queries above, run against a real database
4	learnsql.com, "What Are SQL Window Functions?" (learnsql.com/blog/window-functions)	25 min	Window functions with the history and four worked examples

Chapter 3. Most wrong numbers are right arithmetic on the wrong denominator

Nobody in the diagnostic lost a Section C mark by miscalculating. Marks were lost by dividing the right numerator by the wrong thing, subtracting percentages that multiply, and reading a two-event move as a trend. Reading time: 12 minutes.

Diagnostic questions this chapter revisits, in the paper's order: Q21, Q22, Q23, Q24, Q25, Q26, Q27, Q28; each is worked step by step in Chapter 9.

What you can now do

You can turn a count into a rate before comparing two groups. You can combine two percentage changes without adding them. You can choose between mean and median by looking at the values, not the formula. You can say whether a week's movement is noise from the size of the count. You can turn a recall figure and a base rate into the precision a stakeholder actually wants. You can explain how every part can rise while the whole falls. You can build a cost estimate as a chain with units on every link.

Where this sits

What this chapter covers. Rates against counts, the arithmetic of percentage change, typical values, noise in small counts, base rates, mix shift, and chain estimates. All were worked in the diagnostic; the base-rate item and the mix-shift item are the two the room found hardest, and they get the most space.

Placement. Numbers and reasoning is the third cell of the bottom band. Module 1 uses this reasoning from Week 1 and Module 2 formalises it as evaluation metrics in Weeks 4 to 6.

Outcome tie. The specific moment is the Week 5 exam and the model evaluation that follows it, where precision, recall and the base rate stop being a diagnostic question and become the difference between a fraud model that ships and one that floods the review team.

What was left out. Distributions, confidence intervals and hypothesis tests arrive in Weeks 1 to 2; here you learn to say "that is noise" by inspection, which is what an analyst does before any test.

The picture to remember: the denominator ladder

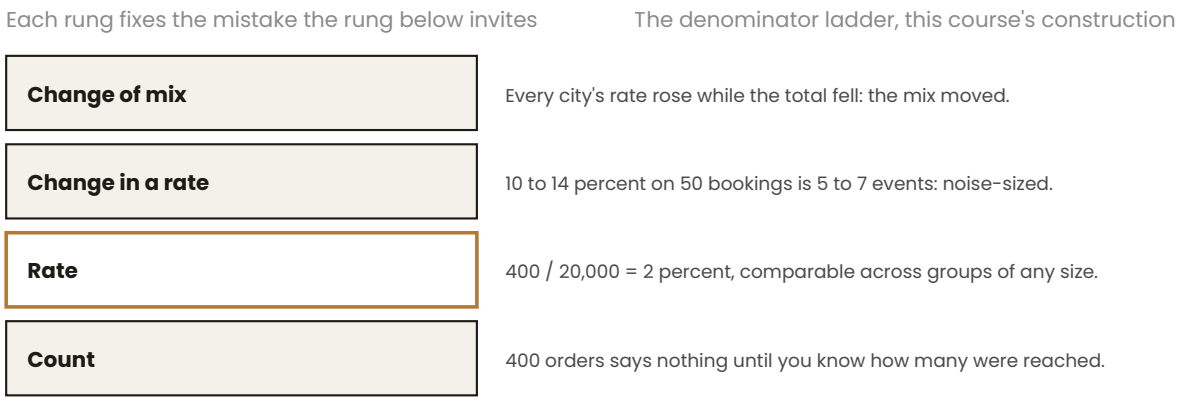


Figure 13. Four rungs, each fixing the mistake the rung below invites. Before any comparison, say which rung you are on. This is this course's own construction; call it the ladder.

Rates, not counts

Q23 put 400 orders beside 150 and asked which group responded better. Four hundred is bigger, and it is also 2 percent of the 20,000 people reached, against 3 percent for the plain email. A count compares nothing until the reach is known; a rate compares groups of any size.

Applied to the thread: Basic orders rose from 60,000 to 64,000 while Basic customers rose from 40,000 to 44,000, so orders per customer went from 1.50 to 1.45. The count went up and the rate went down, and Meera's question is about the rate.

> **WATCH OUT** The tell for a count masquerading as a rate is a comparison with no denominator in the sentence. "Discount customers placed 400 orders" is a count; "2 percent of discount customers ordered" is a rate. Ask for the denominator before agreeing with anything.

Percentage changes multiply

Q21 and Q24 were one fact. Revenue fell 12 percent and orders rose 5 percent, so average order value moved by 0.88 divided by 1.05, about 0.84, a fall of 16 percent; subtracting gives the wrong 7. A 10 percent rise then a 10 percent fall lands at 0.99 of the start; a 50 percent fall needs a 100 percent rise to recover.

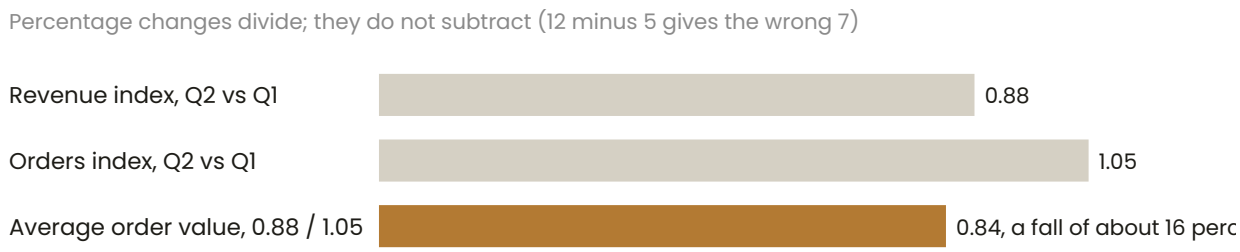


Figure 14. Turn every change into an index (1.00 is the start), then multiply or divide. The subtraction habit survives because it is close enough for small changes and badly wrong for large ones.

Mean, median and what "typical" means

Q22 gave five Plus customers at 300, 350, 400, 450 and 9,000. The mean is 2,100 and describes none of them; the median is 400 and describes four. Look at the values before choosing: one value far from the rest pulls the mean toward itself, and the median ignores how far.

Applied to the thread: average order value for the Plus tier, 360 lakh over 30,000 orders, is a mean, and it is the right mean because it answers "revenue per order", which is what Meera's revenue arithmetic needs. "What does a typical Plus customer spend" is a different question and wants the median. The word "typical" is your cue.

Noise in small counts

Q25 moved a no-show rate from 10 percent to 14 percent on 50 bookings. That is five events becoming seven. Two extra events on a base of fifty sit inside normal week-to-week variation, and the honest reading is "watch several more weeks". The relative jump, 40 percent, is arithmetic on noise.

A working rule, marked as this course's construction: on a count of n events, movements smaller than about the square root of n are the ordinary breathing of the number. Five events breathe by about two; five hundred breathe by about twenty-two. Weeks 1 to 2 give this rule its proper statistical form.

> **IN THE FIELD** The naming of the mix-shift effect in the next section ran on the same small-count logic in reverse: Edward Simpson's 1951 paper worked with contingency tables small enough to check by hand, and it was Colin Blyth who attached Simpson's name to the effect in 1972, half a century after Pearson and Yule first described it (source: Stanford Encyclopedia of Philosophy, Simpson's Paradox, revised June 2026).

Base rates: recall is not precision

Q26 was the item with the largest gap between confidence and correctness. A fraud model catches 90 percent of fraud, flags 5 percent of transactions, and fraud is 1 percent of transactions; of the flagged transactions, only about 18 percent are fraud. The 90 percent is

recall, the share of fraud the model finds; the 18 percent is precision, the share of flags that are fraud; the base rate of 1 percent is what separates them.

1,000 transactions. Fraud is 1 percent; the model catches 90 percent of it and flags 5 percent of everything.

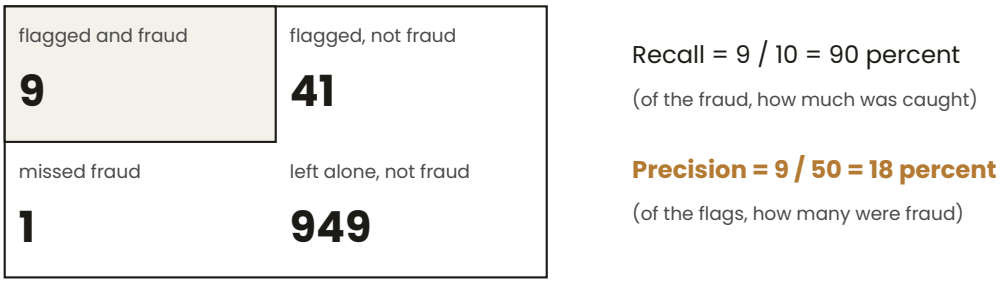


Figure 15. One thousand transactions laid out as four cells. Every question about a classifier is a ratio of two cells, and the base rate sets the cell sizes.

Applied to the thread: when Farhan's ticket classifier reports "90 percent accurate", the first question is the base rate of each label, because a classifier that says "other" for everything is highly accurate on a queue that is mostly "other".

> **CALLBACK** Chapter 4's golden set is how you measure these cells for an LLM classifier; this section is why the measurement has to be reported as precision and recall per label rather than one accuracy number.

Mix shift: every part up, the whole down

Q27 gave a total conversion rate that fell while every city's rate rose. The total is a weighted average of the cities, weighted by traffic, and if traffic moved toward the city with the lower rate, the total falls with no city getting worse.

Every city converts better in Q2, and the total still falls, because more traffic now comes from the low-converting cit

	Q1	Q2
City A	8% of 1,000 = 80	9% of 400 = 36
City B	2% of 1,000 = 20	2.5% of 1,600 = 40
Total	100 / 2,000 = 5%	76 / 2,000 = 3.8%

The total is a weighted average of the cities. Weights moved from A to B; that is the whole story.

Figure 16. Two cities, both improving; the total falls because the weights moved. The question to ask is never "which city got worse" but "where did the traffic go".

Applied to the thread: Kalpa's customer base moved toward Basic and Student in Q2 (55,000 to 62,000 customers, with all the growth in the two cheaper tiers). Even if every tier had held

its average order value, revenue per customer would have fallen. Mix is one of the first three hypotheses on any "the total moved" question.

> **ORIGIN** Karl Pearson described the effect in 1899, Udny Yule in 1903, and Edward Simpson in 1951 in "The Interpretation of Interaction in Contingency Tables"; Colin Blyth named it Simpson's paradox in 1972. It keeps being rediscovered because it is not a paradox of mathematics but of reading a total without its weights (source: Stanford Encyclopedia of Philosophy; Wikipedia, Simpson's paradox).

Chain estimates with units on every link

Q28 asked for a monthly LLM bill from tickets per day, tokens per call and prices per million tokens. The answer, Rs 2,400, was reached by anyone who wrote the chain before the arithmetic; the wrong answers each dropped a link or applied a price to the wrong tokens.



Write the chain before the arithmetic.
A chain with the units on every link cannot slip a factor of ten without you seeing it.

Figure 17. Six links, each with a unit. A chain written this way cannot lose a factor of thirty or apply the output price to input tokens without the error being visible.

Where this shows up in the work

The marketing review. A campaign is declared a success on 400 orders. The reach was 20,000 and the control's rate was higher; the campaign cost the discount for nothing. The analyst who asked for the denominator saved the next quarter's budget.

The fraud model. A model with 90 percent recall goes live and the review team drowns in false alarms, because nobody computed precision at the real base rate. Recall, precision and the base rate on one slide would have set the threshold before launch.

The board deck. Overall conversion fell and every regional lead shows their region improved. Nobody is lying; the mix moved. The analyst who brings the weighted table ends the argument in one minute.

Try this yourself

No-code self-check. (1) A price rises 25 percent then falls 20 percent; where is it? (2) A test with 99 percent recall flags 10 percent of a population where the condition is 1 percent; roughly what share of flags are true? (3) Revenue per customer fell while every tier's revenue per customer rose; name the cause in two words. Key: (1) at the start, since 1.25 times 0.80 is 1.00; (2) about 10 percent, since 0.99 of 1 percent is caught out of 10 percent flagged; (3) mix shift. A miss on (1) sends you to the multiply section, on (2) to base rates, on (3) to mix shift.

Mini project 3, numbers: the tier decomposition. In a notebook in `w00-diagnostic-numbers`, type the Kalpa Q1 and Q2 table from the diagnostic case as a small dictionary. Compute, for each tier, revenue per customer, orders per customer and revenue per order in both quarters, then compute the overall revenue per customer in both quarters. Show, in one printed table, that overall revenue per customer fell more than any single tier's fall would explain, and attribute the difference to mix. Self-check: your three per-tier ratios multiply back to the tier's revenue per customer; the overall figure for Q1 is 804 divided by 55,000; changing the Student customer count changes the overall figure but no tier's own ratios.

Where this gets tested

Interview question. "Conversion fell overall but rose in every segment. What happened?" Tested: mix shift. Strong answer names the weighted average, asks where the traffic moved, and offers to show the weighted table. Weak answer: "one of the numbers must be wrong".

Interview question. "Our model has 95 percent accuracy on fraud. Is that good?" Tested: base rates. Strong answer: at a 1 percent base rate, predicting "no fraud" for everything is 99 percent accurate, so accuracy is the wrong number; ask for precision and recall at the real base rate. Weak answer: "yes, that is high".

Interview question. "A metric moved from 10 to 14 percent this week. Do we act?" Tested: noise in small counts. Strong answer asks for the count behind the percentage and, at 50, calls it noise and proposes a watch window. Weak answer treats the 40 percent relative jump as the finding.

Interview question. "Estimate our monthly spend on this API." Tested: chain estimation. Strong answer writes the chain with units aloud, states which links are assumptions, and gives a range. Weak answer gives one number with no chain.

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
Denominator	The thing a count is divided by to become a rate	Rates section	20,000 people reached
Index	A change written as a multiplier of the start	Multiply section	A 12 percent fall is 0.88
Median	The middle value when sorted	Typical section	400 of 300, 350, 400, 450, 9,000
Base rate	How common the condition is before any test	Base rate section	Fraud at 1 percent of transactions
Precision	Of the flags, the share that were right	Base rate section	9 of 50, 18 percent
Recall	Of the true cases, the share that were caught	Base rate section	9 of 10, 90 percent
Mix shift	A total moving because its weights moved	Mix section	Traffic shifting to a low-converting city

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	3Blue1Brown, "Bayes theorem, the geometry of changing beliefs", youtube.com/watch?v=HZGCoVF3YvM (December 2019)	15 min	The base-rate grid as geometry; watch it twice
2	Google, Machine Learning Crash Course, the Classification module, developers.google.com/machine-learning/crash-course	45 min	Thresholds, the confusion matrix, precision and recall, with interactive exercises
3	Stanford Encyclopedia of Philosophy, Simpson's Paradox, plato.stanford.edu/entries/paradox-simpson	30 min	The history and the worked cases, at a level you can quote in an interview
4	Mini project 3	60 min	The decomposition, in your own notebook

Chapter 4. A language model is a sampler, and a prompt is a specification

Every LLM item in the diagnostic became easy the moment you stopped thinking of the model as a person and started thinking of it as a machine that scores the next token and picks one. Reading time: 11 minutes.

Diagnostic questions this chapter revisits, in the paper's order: Q28, Q33, Q34; each is worked step by step in Chapter 9.

What you can now do

You can explain, in three sentences, how a language model produces text. You can say why the same ticket got two different labels and what to change first. You can write a prompt that names the task, the allowed outputs, one example each and the output shape. You can read the response object and find the text and the token counts. You can measure whether a model's labels can be trusted with thirty hand labels. You can estimate a month's bill before writing a line of code.

Where this sits

What this chapter covers. How text is generated, sampling and temperature, prompts as specifications, the response object and statelessness, the golden set, and token economics. Everything here is worked through Farhan's ticket classifier, which is the Kalpa scenario from Q33 and Q34.

Placement. LLM intuition is the fourth cell of the bottom band. Modules 5 and 6 build on it from Week 10, and Build 4 in Week 12 ships a retrieval assistant that lives or dies by the golden set.

Outcome tie. The specific moment is the Build 4 demo in Week 12, when a panel member asks "how do you know the assistant's answers are right?" and the only acceptable reply is an agreement rate measured on a hand-labelled set.

What was left out. How the model was trained, what attention does inside it, and retrieval over your own documents are Modules 3 to 6. Here the model is a box with a known interface.

The picture to remember: the call anatomy

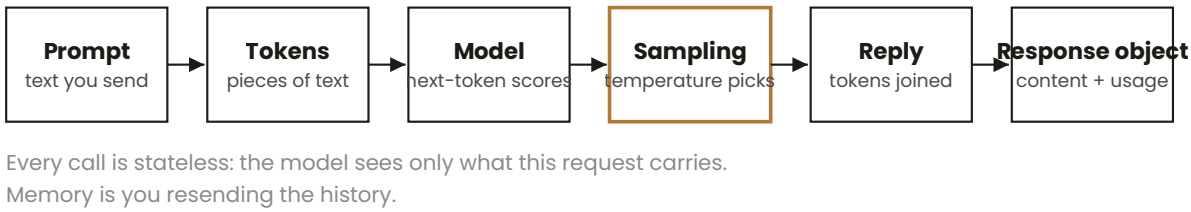


Figure 18. Six stages from your text to the response object. Sampling is ringed because it is the stage that explains Q33. Call this the call anatomy.

> **ORIGIN** The architecture behind every model you will call was published on 12 June 2017 by eight researchers at Google in "Attention Is All You Need"; its original job was translation between English and German, and its base configuration had about 65 million parameters (sources: arXiv 1706.03762, submission history; memx.app, Transformer glossary entry).

How text is generated

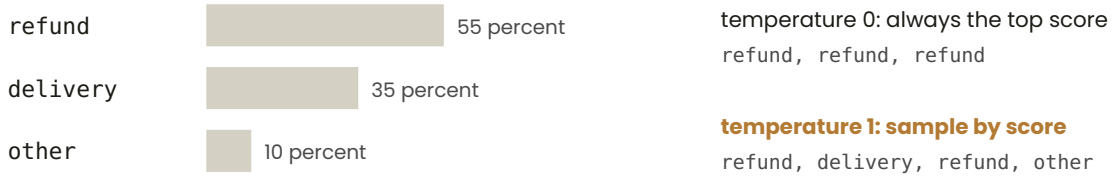
A language model turns your text into tokens, pieces of text a few characters long, and for each position produces a score for every token that could come next. Something picks one token from those scores, appends it, and the loop runs again until a stop token. That is the whole interface: text in, one token at a time out. Everything a model "knows" is in how it scores the next token.

> **IN THE FIELD** Andrej Karpathy's one-hour introduction describes an open model of the time as two files, the parameters and the few hundred lines of code that run them, with the parameters produced by compressing roughly 10 TB of text into about 140 GB; the "dreams" section of the talk shows the model generating plausible but invented product pages, which is the failure the golden set below is designed to catch (source: "[1hr Talk] Intro to Large Language Models", November 2023, youtube.com/watch?v=zjkBMFhNj_g).

Temperature, or why one ticket got two labels

Q33 sent the same ticket twice and got "refund" then "delivery". Nothing changed between the calls except the pick. At temperature 0 the pick is the top-scoring token every time; above 0 the pick is sampled in proportion to the scores, so a token with a 35 percent score wins about one call in three.

The model scores every possible next token. Temperature decides how the pick is made from those scores.



A prompt that never lists the labels adds a second source of variety: the model invents the label set itself.

Figure 19. The same scores, two picking rules. Low temperature makes the output repeatable, and it does not make it right; the scores were already the model's opinion.

Q33's second cause was the prompt: "Label this ticket" never said which labels exist, so the model chose the label set as well as the label. The first fix is the label list; the second is the temperature. Setting temperature to 0 makes replies far more repeatable, and vendors do not promise it makes them identical, so a pipeline that needs the same answer every time still checks the output against the allowed set.

> **WATCH OUT** "The model remembered my first call" is the most common wrong explanation. A call carries no memory of any other call; what you did not send, the model did not see. Conversation history is the caller resending the earlier turns.

A prompt is a specification

The prompt that fixes Q33 has four parts, and each removes one kind of guessing.

A prompt is a specification. The four parts below remove four kinds of guessing.

Task	`Label this support ticket.`
Allowed outputs	`Exactly one of: refund, delivery, other.`
One example each	`'Parcel never came' -> delivery`
Output shape	`Reply with the label only, no sentence.`

Part two is the one Q33 was missing. Without it, two calls on the same ticket can return two labels.

Figure 20. Task, allowed outputs, one example each, output shape. Most bad prompts are missing part two; most unparseable outputs are missing part four.

Applied to the thread: the tickets Farhan's team labels are complaints about the same Q2 that Meera is asking about. A prompt that names the three labels, gives one example each and asks for the label alone turns 1,000 free-text tickets into a column you can count, which is the loop from Chapter 1 applied to a model's output.

> **CALLBACK** Chapter 1, Q11: a JSON example inside a `str.format` template raises `KeyError`. Every prompt you build from a template meets that bug once.

The response object, and statelessness

The response is a dictionary with a list inside it, and reading it is Chapter 1's Q5: `resp["choices"][0]["message"]["content"]` for the text, and a `usage` dictionary with the token counts for the bill. Vendors differ in field names, and the shape is always braces and brackets you can read the same way.

Two rules about state. The model sees only this request, so a "conversation" is the caller sending the whole history every time. And the token count grows with that history, so a long conversation costs more per turn than a short one; that is the arithmetic behind context windows, which Module 5 covers.

The golden set

Q34 asked how to know whether two hundred sentiment ratings can be trusted. Plausible is not evidence; confidence scores are more model output; a second model agreeing is two opinions. The check is a golden set: thirty reviews chosen at random, rated by hand, compared with the model's ratings, giving an agreement rate you can state and defend.

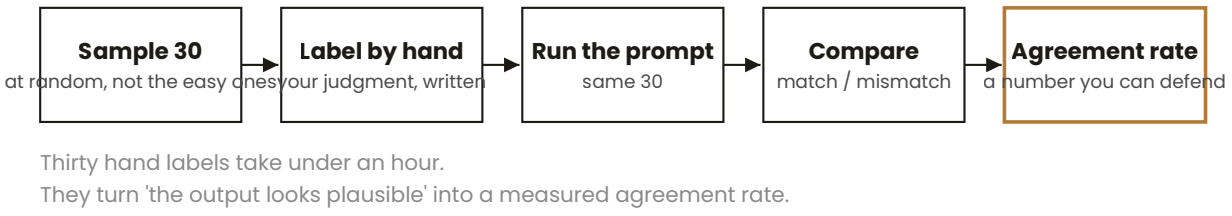


Figure 21. Sample at random, label by hand, run the prompt, compare, report the rate. Thirty items takes under an hour and is the smallest set that says anything.

Report the result per label, as precision and recall, because one accuracy number hides the label the model never predicts. That is Chapter 3's base-rate grid applied to a classifier you did not train.

Token economics

Q28's chain, tickets per day times days times tokens per call times price per million tokens, is the estimate you make before any project is approved. The two prices in the item were given as a supposition; real prices change and are looked up on the day. What does not change is the shape: input and output tokens are priced separately, output is usually several times dearer, and the response's `usage` field is where the real counts come from.

Where this shows up in the work

The classification pipeline. A team labels tickets with "label this" at default temperature and reports 8 percent unparseable outputs. The four-part prompt and a check against the allowed set take the number to near zero in an afternoon.

The assistant demo. The panel asks how you know the answers are right. A team with a golden set says "84 percent agreement on 30 hand-labelled questions, and here are the five misses". A team without one says "it seems to work".

The budget line. Finance asks what the feature will cost per month. The chain with units, plus the `usage` counts from a pilot week, is an answer; a per-call price alone is not.

Try this yourself

No-code self-check. (1) Same prompt, same ticket, two different labels: name the two most likely causes. (2) A model's ratings look right; what is the cheapest check that produces a number? (3) A call costs more on turn ten than on turn one of a conversation; why? Key: (1) temperature above 0 and a prompt with no label list; (2) a hand-labelled sample and an agreement rate; (3) the whole history is resent, so the input token count grows. A miss on (1) sends you to temperature, on (2) to the golden set, on (3) to statelessness.

Mini project 4, language models: a golden set without an API key. In `w00-diagnostic-llm`, write twenty short support tickets in a text file, label each by hand as refund, delivery or other, and record your labels in a CSV. Then, using any chat assistant you already have access to, paste the four-part prompt from Figure 20 with the twenty tickets and record the model's labels in a second column. Compute agreement overall and per label in a notebook, and write the three worst misses in the README with one sentence each on why the model went wrong. Self-check: the CSV has twenty rows and no empty labels; the per-label counts add up to twenty; the README names the label the model over-predicts.

Where this gets tested

Interview question. "Explain how an LLM generates text." Tested: the call anatomy. Strong answer: tokens in, next-token scores out, one token picked and appended per step, stateless per call. Weak answer: "it understands the question and answers".

Interview question. "Why do I get different answers to the same prompt?" Tested: sampling. Strong answer: temperature above 0 samples from the scores; lower it and constrain the output, and still validate against the allowed set. Weak answer: "the model is random".

Interview question. "How would you evaluate an LLM classifier before shipping it?" Tested: the golden set. Strong answer: a random hand-labelled sample, precision and recall per label, and the misses reviewed. Weak answer: "look at some outputs".

Interview question. "What does temperature 0 guarantee?" Tested: honesty about limits. Strong answer: far more repeatable output, not a guarantee of identical output, so pipelines still validate. Weak answer: "deterministic output".

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
Token	A piece of text the model reads and writes	Generation section	"refund" may be one token; "unparseable" may be three
Temperature	How much the pick is spread across the scores	Temperature section	0 picks the top score; 1 samples by score
Stateless	A call sees only what it was sent	Response section	History is resent each turn
Golden set	Hand-labelled items used to measure a model	Golden set section	Thirty random reviews, rated by you
Agreement rate	Share of items where model and hand label match	Golden set section	25 of 30, 83 percent
Usage	The response field with token counts	Response section	`prompt_tokens: 412`

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	3Blue1Brown, "But what is a GPT? Visual intro to transformers", youtube.com/watch?v=wjZofJX0v4M (April 2024)	27 min	Tokens, scores and sampling drawn, not asserted
2	Andrej Karpathy, "[1hr Talk] Intro to Large Language Models", youtube.com/watch?v=zjkBMFhNj_g (November 2023)	60 min	The two files, training against inference, and the failure modes
3	Anthropic, prompt engineering overview, platform.claude.com/docs/en/build-with-claude/prompt-engineering	30 min	The four-part prompt as the vendor states it, with examples
4	OpenAI, prompt engineering guide, platform.openai.com/docs/guides/prompt-engineering	20 min	The same ideas in a second vendor's words, which is how you learn what is general
5	Mini project 4	60 min	Your first golden set

Chapter 5. An unfamiliar business problem is decomposed, never solved in one move

Section D of the diagnostic contained no arithmetic harder than division, and it separated the room more than any other section, because it tested the order in which you look at a table. Reading time: 9 minutes.

Diagnostic questions this chapter revisits, in the paper's order: Q29, Q30, Q31, Q32, Q33, Q34, Q35, Q36, Q37, Q38, Q39, Q40; each is worked step by step in Chapter 9.

What you can now do

You can read a table for its shape before computing on it. You can split a total into the parts that moved and name the one that carries the change. You can test a hypothesis against a table instead of reaching for more data. You can choose the fairest comparison available. You can name the check that protects a finding before it goes in front of a decision maker. You can act well in the six situations from Section E because you can say the rule they share.

Where this sits

What this chapter covers. Reading a table, the driver question, hypothesis against evidence, the fair comparison, the window check, and the four habits behind the six judgment situations. All of it runs on Meera's table.

Placement. Business problems is the fifth cell of the bottom band. Build 1 in Week 3 is exactly this chapter with real data, and the business case discussions in every build week score it.

Outcome tie. The specific moment is the capstone panel defence in Week 20, where the first question after your demo is "so what changed for the business", and the answer is a sentence built the way this chapter builds it.

What was left out. Experiment design, which makes the fair comparison rigorous, arrives in Module 2; stakeholder writing arrives with the Week 16 solution proposal.

The picture to remember: the decomposition tree

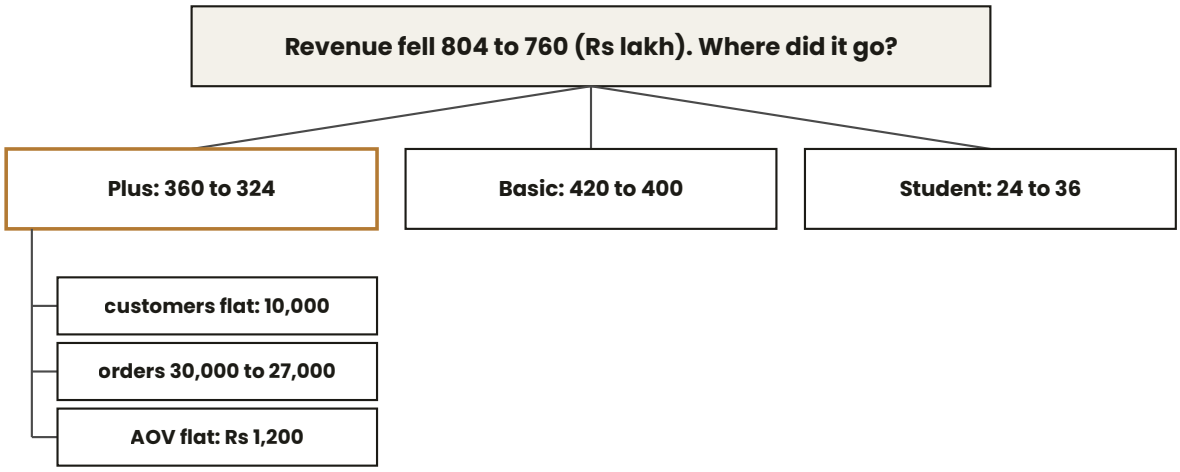


Figure 22. Total at the top, the parts that moved below, the driver under the part that moved most. Every "where did it go" question is this tree; call it the tree.

Read the table before computing on it

Q29's table had six numbers per tier, and the fall was visible before any division: Plus lost 36 lakh, Basic lost 20, Student gained 12, net 44 down. Reading the deltas first tells you where to compute. The Plus column then shows customers flat at 10,000, orders down 10 percent, and revenue per order flat at Rs 1,200, so the driver is frequency, not price and not churn.

> **WATCH OUT** The instinct to compute a ratio before reading the deltas is the instinct that produced the wrong answers to Q29. A ratio on the wrong tier is precise and useless. Read down the columns, circle the biggest delta, then compute inside that column.

Hypothesis against evidence

Q30 offered "Plus customers are downgrading to Basic" as Kavya's first hypothesis. The table already tests it: Plus customers were 10,000 in both quarters. Downgrades would have lowered that count unless upgrades exactly replaced them, which the table gives no reason to believe. The hypothesis is weakened by the evidence in front of you, and the reflex to ask for churn data is a reflex to avoid reading.

The habit, marked as this course's construction: for every hypothesis, write the one number in the table that would move if it were true, then look at it.

> **CALLBACK** Chapter 3's mix-shift section is the second hypothesis for this table: the customer base moved toward the cheaper tiers. It is tested the same way, by the customer counts per tier.

The fair comparison

Q31 asked whether the monsoon discount caused the rise in Basic orders. Before-and-after compares the discount plus the season plus the festival calendar with nothing; the fair comparison holds everything else still and varies only the discount: Basic customers who received it against Basic customers who did not, over the same weeks.

The fairest comparison is the one where only the thing you are testing differs

Comparison	What else changed between the two sides	Fair?
Basic Q2 vs Basic Q1	season, festivals, days, the discount	no
Basic Q2 vs Plus Q2	the tier itself	no
Discounted vs non-discounted Basic, same weeks	only the discount	yes

Figure 23. Three candidate comparisons and what else each one lets vary. Fairness is the shortness of the "what else changed" column.

> **IN THE FIELD** Google's engineering education team taught its Machine Learning Crash Course to more than 18,000 of its own engineers before publishing it in March 2018; the course's classification module is where the fair-comparison habit becomes a held-out evaluation set, which is the same idea applied to a model rather than a discount (source: Google Developers blog, "Machine Learning Crash Course", 2 March 2018).

The window check

Q32 asked which check protects the Plus finding most. Not the marketing budget, not re-adding the sums, not the share of revenue: the check is whether Q1 and Q2 had the same number of trading days and whether a festival sat in one quarter only. A 3 percent difference in trading days is a 3 percent difference in revenue with nothing else changed, and a festival can move a tenth of a quarter's orders on its own.

The window check runs before every "X fell" sentence you will ever say to a decision maker. It costs one calendar lookup.

Applied to the thread, the sentence Meera hears is now buildable: "Revenue fell 44 lakh from Q1 to Q2. Thirty-six of it is the Plus tier, where the same 10,000 customers ordered 10 percent less often at the same order value; the quarters had the same trading days and no festival difference; the first thing to test is Plus frequency, and the discount's effect on Basic needs a like-for-like comparison before we credit it."

The six situations, and the four habits behind them

Section E asked for the best and worst action in six situations. They reduce to four habits, and the habits are the reason the section was in a technical diagnostic: an analyst who cannot do these is not employable, whatever the code looks like.

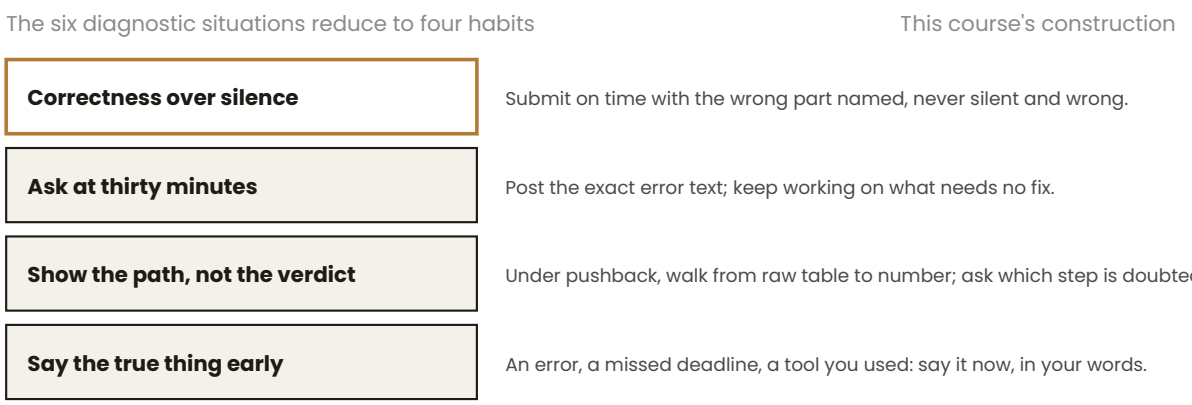


Figure 24. Q35 to Q40 as four habits. Say the true thing early covers the board figure, the AI-assisted answer and the criticised chart; show the path covers Anand's pushback; ask at thirty minutes covers the environment error; correctness over silence covers the deadline. This is this course's own construction.

The worst actions across all six had one shape: a silence, a delay or a pretence that made the next conversation harder. The best actions had one shape too: they made the problem visible sooner and cheaper. When in doubt, choose the option that lets someone else check you.

Where this shows up in the work

The first Finance meeting. Anand says the analysis cannot be right. The path from raw table to number, shown line by line, either finds his objection or ends it. Changing the conclusion to please him costs the whole analysis; escalating costs the relationship.

The build week presentation. "Revenue fell because of the discount" is a hypothesis presented as a finding. The panel asks for the fair comparison, and a group that ran it before the presentation keeps its credibility.

The board figure. A 4 percent error you found yourself, disclosed within the hour with cause and size, is a footnote. The same error found by someone else a week later is a trust event.

Try this yourself

No-code self-check. (1) Total up, every segment down: possible? (2) Which single number tests "customers are leaving"? (3) Before saying "orders fell 8 percent", what do you check? Key: (1) yes, by mix shift; (2) the customer count per period; (3) that the two windows have the same length and no one-sided event. A miss on (1) sends you to Chapter 3's mix section, on (2) to hypothesis against evidence, on (3) to the window check.

Mini project 5, business problem: the one-paragraph answer. In `w00-diagnostic-business`, write a README with three parts. Part one is the decomposition tree for Meera's table as a nested list, with the delta beside every node. Part two is a table of three hypotheses, the one number in the table that tests each, and the verdict. Part three is the sentence Meera hears, in your own words, followed by the check you would run first. Self-check: every node's deltas

add up to its parent's; every hypothesis names a number that already exists in the table; the sentence contains a number, a driver and a check, and no word you would not say aloud.

Where this gets tested

Interview question. "Revenue fell 5 percent. Walk me through what you would do." Tested: the tree. Strong answer: split by segment, find the part that carries the fall, decompose it into customers, frequency and order value, check the window, then hypothesise. Weak answer: "I would look at the data" or a list of possible causes with no order.

Interview question. "Your stakeholder disagrees with your number. What do you do?" Tested: show the path. Strong answer: walk from raw table to number and ask which step is doubted; change the number only if a step is wrong. Weak answer: "explain it again more clearly" or "escalate".

Interview question. "How would you know if the discount worked?" Tested: the fair comparison. Strong answer: recipients against non-recipients in the same weeks, and what could still differ between them. Weak answer: before-and-after.

Interview question. "You find an error in a figure you already sent. What now?" Tested: say the true thing early. Strong answer: send the correction now, with cause and size, to the person who received it. Weak answer: fix it quietly for next time.

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
Delta	The change in a number between two periods	Reading section	Plus revenue 360 to 324, delta 36
Driver	The component whose movement carries the change	Reading section	Plus order frequency
Hypothesis	A claim about cause that a number can test	Hypothesis section	"Plus customers are downgrading"
Fair comparison	Two groups that differ only in the thing tested	Comparison section	Discounted against non-discounted Basic
Window check	Confirming two periods are comparable in length and events	Window section	Same trading days, no one-sided festival
Like for like	The same thing measured the same way in both periods	Comparison section	Same weeks, same tier

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	Barbara Minto, The Pyramid Principle (the book that codified the answer-first structure used in consulting; any edition)	3 h for the first three chapters	The sentence Meera hears is a Minto sentence: answer, then the support
2	Google, Machine Learning Crash Course, Classification module, developers.google.com/machine-learning/crash-course	45 min	The fair comparison as a held-out set
3	Mini project 5	60 min	Your first one-paragraph answer

Chapter 6. A DataFrame is the SQL pipeline with a different spelling

pandas was not in the diagnostic, and it is the tool that replaces most of Chapter 1's loops from Week 1 onward. It takes an afternoon to learn once you see that every move has a SQL twin. Reading time: 7 minutes.

What you can now do

You can load a CSV into a DataFrame and say what each column's type is. You can turn text amounts into numbers with the bad ones marked, not dropped. You can filter rows with a boolean mask. You can group by a column and aggregate. You can merge two tables and check the row count before and after. You can write a notebook whose last cell asserts the answer.

Where this sits

What this chapter covers. The DataFrame and Series, loading and cleaning, filtering and grouping, merging, and the assert-cell habit. Plotting and dates are mentioned and worked in Week 1.

Placement. pandas is the sixth cell of the bottom band. Every Module 1 notebook uses it, and Build 1 in Week 3 is built in it.

Outcome tie. The specific moment is the Week 1 notebook where the room computes revenue by tier for Kalpa in six lines of pandas after having written it as a thirty-line loop the day before.

What was left out. Time series, pivoting and plotting arrive in Week 1; performance on large data arrives when it matters, in Module 7.

The picture to remember: the DataFrame

A DataFrame is a table with an index; each column is a Series with one dtype. Most bugs are dtype bugs.

index	order_id	tier	amount	status
0	1	Plus	1200.0	paid
1	2	Basic	NaN	paid
2	3	Plus	800.0	cancelled

`df['amount']`
is a Series, dtype float64

NaN is pandas' missing value; sum() skips it, like SQL's SUM

Figure 25. A table with an index, each column a Series with one type. The ringed column is a float Series with a missing value, which is where most bugs live.

> **ORIGIN** Wes McKinney started building pandas in 2008 at AQR Capital Management, a quantitative investment firm, because Python had no tool for the tabular analysis he needed, and persuaded AQR to open-source it in 2009; the project became a fiscally sponsored NumFOCUS project in 2015 (sources: Wikipedia, pandas (software); O'Reilly, Learning Pandas, chapter 1).

DataFrame and Series

A DataFrame is a table; each column is a Series; every Series has one dtype, and the dtype decides what arithmetic means, exactly as Chapter 1's types did. `df.dtypes` is the first thing to print after loading anything. A column that should be numbers and reads as `object` is a column of strings, and every sum on it will be wrong or will fail.

Load and clean: the bad values are marked, not dropped

`pd.read_csv("orders.csv")` gives strings where the file had text amounts like `'1,200'`. The conversion is one line: `pd.to_numeric(df["amount"].str.replace(",", ""), errors="coerce")`. Values that cannot convert become `NaN`, pandas' missing value, and stay in the table where you can count them. That is Chapter 1's `try` per row, applied to a whole column at once, with the rejects kept as `NaN` instead of a list.

`NaN` behaves like SQL's NULL for aggregation: `sum()` and `mean()` skip it, `count()` does not count it, and `df["amount"].isna().sum()` tells you how many rows failed conversion, which is the number Anand asked for in Q9.

Filter and group: the pipeline again



The same six moves as the SQL pipeline, in the same order. Learn one and you have learned both.

Figure 26. Six moves that mirror the SQL pipeline. The mask is `WHERE`, `groupby` is `GROUP BY`, `agg` is the `SELECT` list, `merge` is `JOIN`.

Applied to the thread, Meera's revenue by tier is:

```
paid = df[df["status"] == "paid"]
by_tier = paid.groupby("tier").agg(orders=("order_id", "count"), revenue=("amount",
"sum"))
by_tier["aov"] = by_tier["revenue"] / by_tier["orders"]
```

The mask `df["status"] == "paid"` is a Series of True and False; indexing with it keeps the True rows. `groupby("tier")` collapses to one row per tier; `agg` names the output columns. The division on the last line is true division, so `aov` is a float and Chapter 1's Q7 cannot happen.

> **CALLBACK** Chapter 2's pipeline figure is this chain. If a pandas expression confuses you, write the SQL it corresponds to, and the order becomes clear.

Merge, and the fan-out again

`paid.merge(customers, on="customer_id", how="left")` is Chapter 2's `LEFT JOIN`. The fan-out from Q20 happens here too, silently. Print `len()` before and after, and use `validate="many_to_one"` on the merge, which raises an error the moment the right-hand table has duplicate keys instead of quietly multiplying your rows.

> **WATCH OUT** `df2 = df` in pandas is Chapter 1's board: two names, one object. A DataFrame you meant to keep untouched changes when you modify the "copy". Use `df.copy()` when you want a separate object.
> **IN THE FIELD** The pandas user guide's own first stop for new users is a page called "10 minutes to pandas", and its opening moves are exactly the ones in this chapter: build a Series and a DataFrame, look at `dtypes`, select, filter, group and merge (source: pandas.pydata.org, User Guide).

The assert cell

Every notebook you submit ends with a cell that checks the answer: `assert abs(by_tier["revenue"].sum() - paid["amount"].sum()) < 1e-6` proves the group totals add up to the raw total, and `assert len(merged) == len(paid)` proves the merge did not fan out. A notebook that ends with an assertion is a notebook a reviewer can trust without reading it.

Where this shows up in the work

The loop that took a day. A colleague's thirty-line loop over rows is replaced by three lines and runs in a second on a million rows. The tell is a `for` loop over `df.iterrows()`.

The silent duplication. A merge doubled the revenue column and nobody noticed until Finance did. `validate="many_to_one"` would have raised on the first run.

The object column. A sum over an `object` column concatenated strings and produced a number with nine digits. `df.dtypes` after load is a ten-second habit.

Try this yourself

No-code self-check. (1) A column of amounts loads as `object`; what happened? (2) Which pandas argument turns `'n/a'` into `NaN` instead of raising? (3) Rows doubled after a merge; which argument would have caught it? Key: (1) at least one value is text, so the whole column is strings; (2) `errors="coerce"`; (3) `validate="many_to_one"`. A miss on (1) sends you to dtypes, on (2) to load and clean, on (3) to merge.

Mini project 6, pandas: mini project 1 again, in six lines. In `w00-diagnostic-pandas`, save the ten rows from mini project 1 as `orders.csv`. Load it, convert `amount` with `to_numeric(errors="coerce")`, print the count of `NaN`, compute revenue and order count by tier for paid orders, merge a two-row `customers.csv` with `validate="many_to_one"`, and end

with two assert cells. Self-check: the `NaN` count equals the number of bad amounts you planted; the tier totals equal mini project 1's totals; the notebook runs top to bottom after a restart.

Where this gets tested

Interview question. "How do you handle a numeric column that has some bad strings?" Tested: cleaning without dropping. Strong answer: `to_numeric(errors="coerce")`, then count and inspect the `NaN` rows before deciding anything. Weak answer: "drop the bad rows".

Interview question. "What is the difference between `groupby().sum()` and a window function?" Tested: output shape. Strong answer: `groupby` collapses to one row per key; `transform` keeps every row with the group value attached, which is the pandas window. Weak answer: no distinction.

Interview question. "Your merge produced more rows than you expected. Why?" Tested: fan-out. Strong answer: duplicate keys on the many side; `validate` catches it and aggregating first fixes it.

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
DataFrame	A table with an index and typed columns	The picture	`pd.read_csv("orders.csv")`
Series	One column, with one dtype	The picture	`df["amount"]`
dtype	The type of a column	DataFrame section	`float64`, `object`
NaN	pandas' missing value	Load section	The result of `to_numeric` on `'n/a'`
Boolean mask	A True/False Series used to select rows	Filter section	`df["status"] == "paid"`
validate	A merge argument that checks key uniqueness	Merge section	`validate="many_to_one"`

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	pandas User Guide, "10 minutes to pandas", pandas.pydata.org/docs/user_guide/10min.html	30 min	The official first stop, in the order this chapter used
2	Corey Schafer, Python Pandas Tutorial (Part 1): Getting Started, youtube.com/watch?v=ZyhVh-qRZPA, and the rest of the playlist, youtube.com/playlist?list=PL-osiE80TeTsWmV9i9c58mdDCSskIFdDS	25 min for part 1, then parts 4 and 8	Filtering and grouping shown on a real dataset
3	Mini project 6	90 min	Mini project 1 rebuilt in pandas, with the asserts

Chapter 7. A PivotTable is GROUP BY for people who will never see your code

Excel is secondary in Week 0 and unavoidable at work: every number you compute will be checked by someone in a spreadsheet, and the fastest way to be trusted is to be able to rebuild your answer where they can see it. Reading time: 5 minutes.

What you can now do

You can turn a range into a Table so formulas stay correct as rows are added. You can build a PivotTable that reproduces a `GROUP BY`. You can write `SUMIFS` and `COUNTIFS` for a filtered total. You can look up a value from another table with `INDEX` and `MATCH`. You can say where Excel stops and a notebook starts.

Where this sits

What this chapter covers. Tables, PivotTables, conditional totals, lookups, and the limits. It is short because the operations are Chapter 2's with different spelling, and the figure below is the whole map.

Placement. Excel is the seventh cell of the bottom band and a floating slot in Week 0 if time permits. It reappears whenever a stakeholder wants to check a number.

Outcome tie. The specific moment is the Build 1 presentation in Week 3, when a panel member opens the group's numbers in a spreadsheet and asks why the pivot disagrees with the notebook. It should not, and this chapter is how you make sure.

What was left out. Charts, Power Query and the Data Model are not needed in this programme's first weeks.

The picture to remember: one question, three tools

One question, three tools: the operation is the same; only the spelling changes

Question	SQL	pandas	Excel
Revenue per tier	GROUP BY tier	groupby('tier').sum()	PivotTable, or SUMIFS
Paid orders only	WHERE status='paid'	df[df.status=='paid']	filter, or SUMIFS criteria
Latest call per ticket	ROW_NUMBER() OVER	sort + drop_duplicates	sort, then remove duplicates
Attach city to orders	JOIN customers	merge(customers)	INDEX and MATCH, or XLOOKUP

Figure 27. Four questions from Meera's table, answered in SQL, pandas and Excel. The operation is identical; only the spelling changes.

Tables, not ranges

Select the data and press Ctrl+T, or Insert then Table. A Table has a name, its formulas fill every row, and a PivotTable built on it grows with it. A range does none of that, and most "the total is wrong" complaints in spreadsheets are a range that stopped one row short.

The PivotTable is GROUP BY

Insert then PivotTable, put `tier` in Rows, `amount` in Values, `status` in Filters set to paid. That is `SELECT tier, SUM(amount) FROM orders WHERE status = 'paid' GROUP BY tier`. Drag `quarter` to Columns and you have the cross-tab Meera's table was drawn from.

> **IN THE FIELD** Microsoft's own guide to creating a PivotTable applies to Excel for Microsoft 365 through Excel 2016 and walks the same four steps: select the cells, Insert then PivotTable, choose the placement, tick the fields (source: support.microsoft.com, "Create a PivotTable to analyze worksheet data").

SUMIFS and COUNTIFS

A pivot is a report; a formula is a single cell you can point at. `=SUMIFS(amount, tier, "Plus", status, "paid")` is the Plus revenue, and `=COUNTIFS(tier, "Plus", status, "paid")` is the order count; divide them for the average order value. Column names in these formulas are the Table's column names, so they survive sorting and new rows.

> **WATCH OUT** `SUMIF` with one criterion and `SUMIFS` with several take their arguments in a different order. Use `SUMIFS` everywhere, even for one criterion, and the order is always range, criteria range, criteria.

INDEX and MATCH

To attach a customer's city to an order, `=INDEX(customers[city], MATCH([@customer_id], customers[customer_id], 0))` finds the row in the customers table and returns the city. `XLOOKUP` does the same in one function on current Excel; `INDEX` and `MATCH` work everywhere, including the LibreOffice this programme's workbooks are checked in. A lookup that finds two rows returns the first and says nothing, which is Chapter 2's fan-out in a spreadsheet.

Where Excel stops

Above about a million rows the sheet will not hold the data. Above about ten steps, a spreadsheet cannot be reviewed, because the steps are hidden inside cells. And a spreadsheet cannot be rerun on next month's file without hand-editing. Those three limits are where the notebook and the query begin, and the programme's rule is simple: compute in the notebook or the database, then rebuild the one headline number in the sheet so the stakeholder can see it agree.

Try this yourself

No-code self-check. (1) Your `SUM` misses the last row; what did you skip? (2) The pivot's Plus revenue disagrees with your notebook; name two causes. (3) Which lookup pattern works in both Excel and LibreOffice? Key: (1) making the data a Table; (2) cancelled orders included in one and not the other, or amounts stored as text in the sheet; (3) `INDEX` and `MATCH`. A miss on (1) sends you to Tables, on (2) to the pivot, on (3) to lookups.

Hands-on, forty-five minutes. Paste mini project 1's rows into a sheet, make them a Table, build the pivot for revenue by tier, write the `SUMIFS` for Plus, and check the two agree with your notebook. Put a screenshot of the pivot in the `w00-diagnostic-pandas` README beside the pandas result.

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
Table	A named range whose formulas and pivots grow with it	Tables section	Ctrl+T on the orders range
PivotTable	A drag-and-drop `GROUP BY`	Pivot section	tier in Rows, amount in Values
SUMIFS	A sum with one or more conditions	Formulas section	Plus revenue, paid only
INDEX and MATCH	A lookup by position found by value	Lookup section	City for a customer id

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	Microsoft Support, "Create a PivotTable to analyze worksheet data", support.microsoft.com/office/18fb0032-b01a-4c99-9a5f-7ab09edde05a	15 min, with the embedded video	The vendor's own steps, platform by platform
2	The hands-on above	45 min	The pivot that agrees with your notebook

Chapter 8. Your GitHub is the portfolio interviewers actually open

You created the account this week. This chapter is the difference between an account and a portfolio: a profile that says who you are, one repository per project with a README that proves the code ran, a public record of your thinking in Discussions, and a weekly loop that adds to all three. Reading time: 13 minutes.

What you can now do

You can pick a username you will not regret and lock the account with two-factor authentication. You can publish a profile README. You can create a repository, commit a file in the browser, and open the same repository in a Codespace with VS Code, Python and a terminal ready. You can answer an exercise on GitHub Discussions in the form the programme expects. You can lay out twenty weeks of work so a recruiter finds any piece in seconds.

Where this sits

What this chapter covers. Account and security, the profile README, repositories and commits, Codespaces as the programme's single environment, Discussions etiquette, the portfolio layout and the weekly loop. Branches, pull requests and the Git command line arrive in Week 1 and are not needed to start.

Placement. GitHub is the eighth cell of the bottom band and the surface everything else lands on: exercises and submissions live on GitHub Discussions and the LMS, and every notebook runs in Codespaces so nothing installs locally.

Outcome tie. The specific moment is the first interview around Week 15, when the interviewer opens your profile on their phone while you are still introducing yourself.

What was left out. Git branching, merge conflicts and pull request review are Week 1; GitHub Actions and Pages are later, when you have something to automate or publish.

The picture to remember: the account anatomy

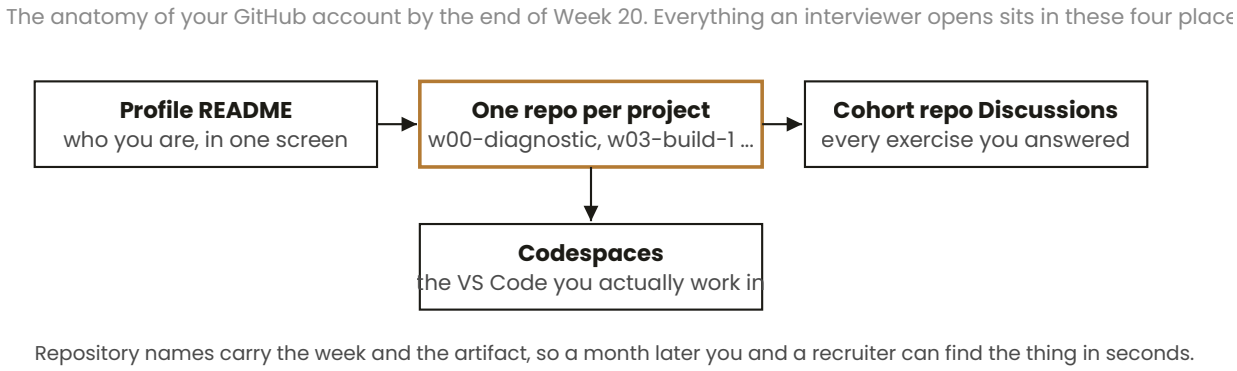


Figure 28. Four places an interviewer looks, connected by the Codespace you work in. Call this the anatomy; the portfolio section fills it in.

> **ORIGIN** Git was written by Linus Torvalds in April 2005 to manage the Linux kernel after the previous tool's licence changed, and GitHub was launched on 10 April 2008 by Tom Preston-Werner, Chris Wanstrath and PJ Hyett as a hosted, social layer over it; Microsoft acquired GitHub in June 2018, and by the mid-2020s the platform reported more than 100 million users (sources: noze.it, GitHub founding; Britannica, GitHub).

The account, the username and the lock

The figures in this chapter are illustrations drawn to the current layout of the screens, and each is paired with the GitHub Docs page that carries the real screenshot, since the interface changes and the documentation moves with it.

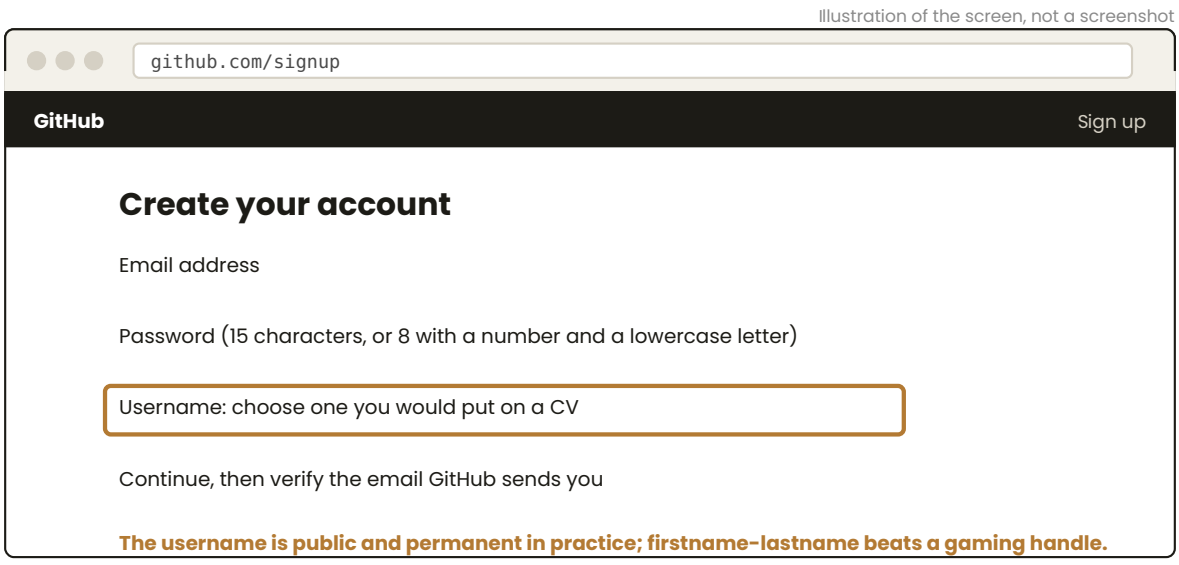


Figure 29. The sign-up form. The username is public, shows in every repository address and is what a recruiter types; a real-name form is the safe choice.

Go to github.com, choose Sign up, enter an email you will still own in five years, a password, and a username. Verify the email from the message GitHub sends; without a verified email you cannot create a repository (source: docs.github.com, Creating an account on GitHub). If your current username is a handle you would not put on a CV, change it now in Settings, before any repository links exist.

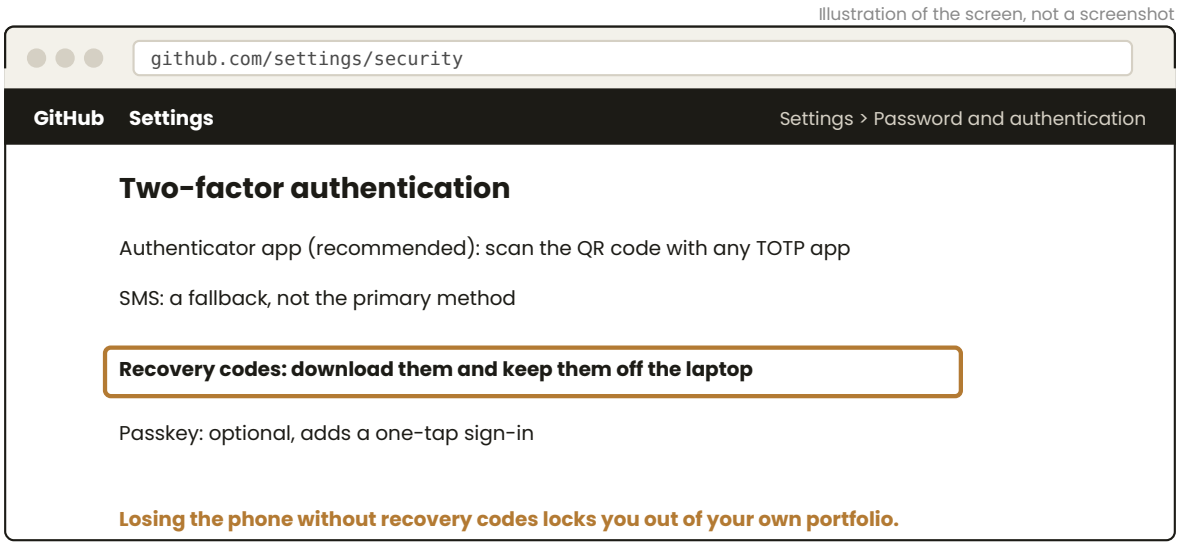


Figure 30. Settings, then Password and authentication. The ringed step is the one people skip, and it is the one that recovers the account when the phone is lost.

Turn on two-factor authentication with an authenticator app, download the recovery codes, and store them somewhere that is not the laptop. GitHub Docs, "Configuring two-factor authentication", has the current screenshots for the app you choose.

The profile README

A repository with the same name as your username, public, containing a `README.md`, is shown at the top of your profile (source: docs.github.com, Managing your profile README). Create it now with four lines: who you are in one sentence, what you are building on this programme, the three things you want to be good at by Week 20, and a link to your best repository. Rewrite it in Week 10 and Week 20. GitHub's own docs page for this is docs.github.com/en/account-and-profile/how-tos/profile-customization/managing-your-profile-readme, and it shows the exact New repository dialog.

> **WATCH OUT** A profile README that lists twenty technologies you have not used reads as noise to anyone who has hired before. Three things you can demonstrate, each linked to a repository, beats a wall of badges.

The first repository and the first commit

Illustration of the screen, not a screenshot

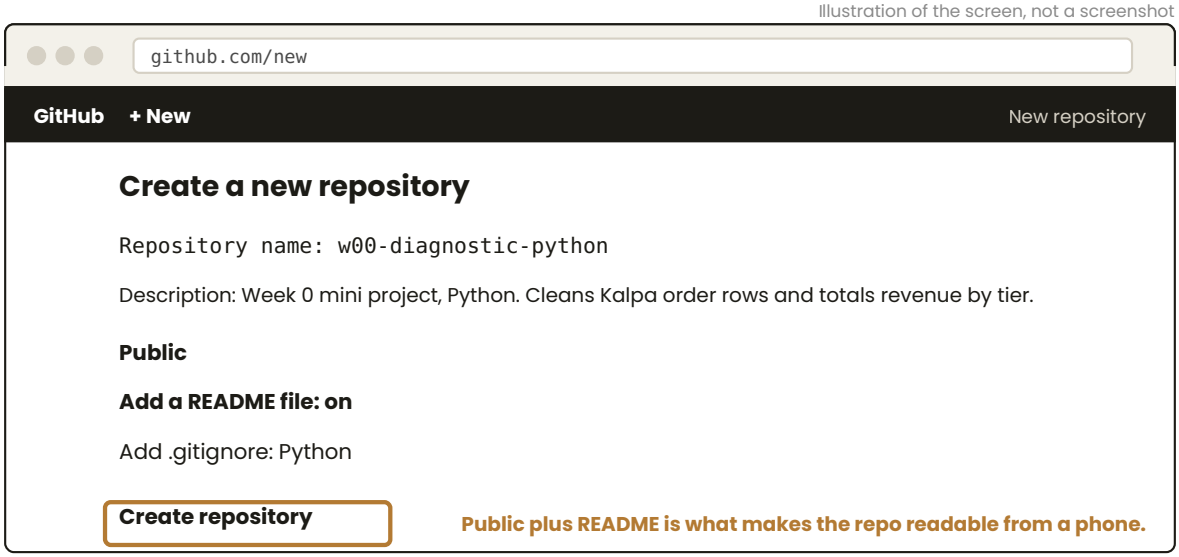


Figure 31. The New repository form. Public plus a README is what makes the repository readable from a phone before any code exists.

Click the plus in the top right, then New repository. Name it `w00-diagnostic-python`, add the one-line description, keep it Public, tick Add a README file, choose the Python `.gitignore`, and create it. Then open `README.md`, click the pencil to edit, replace the text with the five headings from Figure 32, and choose Commit changes with a message such as "Add README skeleton". That is a commit: a saved, named, dated change. Everything you do on GitHub from now on is a series of these.

A project README that a recruiter reads in twenty seconds. Five headings, in this order, every repo.

```
# w00-diagnostic-python

One sentence: what this does and for whom (Kalpa Retail's Q1 to Q2 revenue question).

## What it shows (the concept, in your words)

## How to run (open in Codespaces, run notebook.ipynb, top to bottom)

## Result (one table or one number, pasted)

## What I would do next (one honest limit)
```

The Result heading is the one most people skip, and it is the only one that proves the code ran.

Figure 32. The five-heading README every project repository carries. The Result heading is the one that proves the code ran.

Codespaces: the programme's single environment

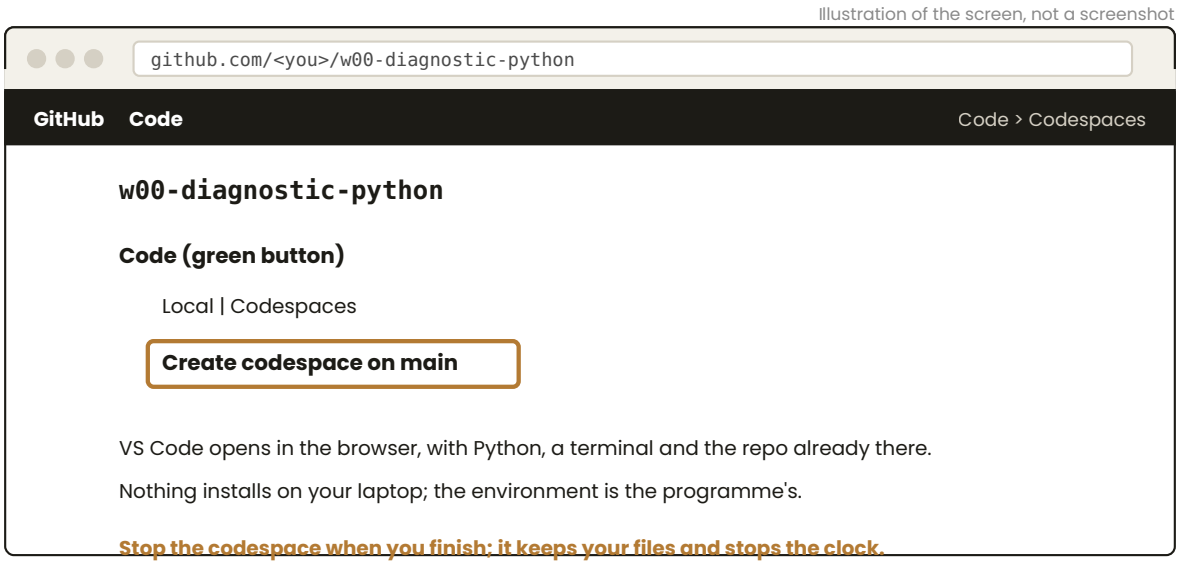


Figure 33. The green Code button, the Codespaces tab, and Create codespace on main. VS Code opens in the browser with the repository already checked out.

On the repository page, click Code, then the Codespaces tab, then Create codespace on main. In under a minute you have VS Code in the browser with Python, a terminal and the files. Create `notebook.ipynb`, run a cell, and when you are done open the Source Control view, write a message, and commit and push; or simply close the tab, since the codespace keeps your work until you delete it. Stop the codespace when you finish a session to preserve your free hours. The quickstart at docs.github.com/en/codespaces/getting-started/quickstart is the canonical walkthrough, and GitHub Skills' "Code with GitHub Codespaces and Visual Studio Code" (github.com/skills/code-with-codespaces) is a guided exercise that takes under an hour.

> **IN THE FIELD** GitHub's own Skills course for Codespaces is a public repository you copy into your account and complete by making commits; its automation checks each step and moves you to the next, which is the same commit-and-verify loop this programme's take-homes use (source: github.com/skills/code-with-codespaces).
> **CALLBACK** Chapter 1's rule that a notebook is proven only by restart and run-all is what you do in the Codespace before every commit that touches a notebook.

Discussions: public writing, on purpose

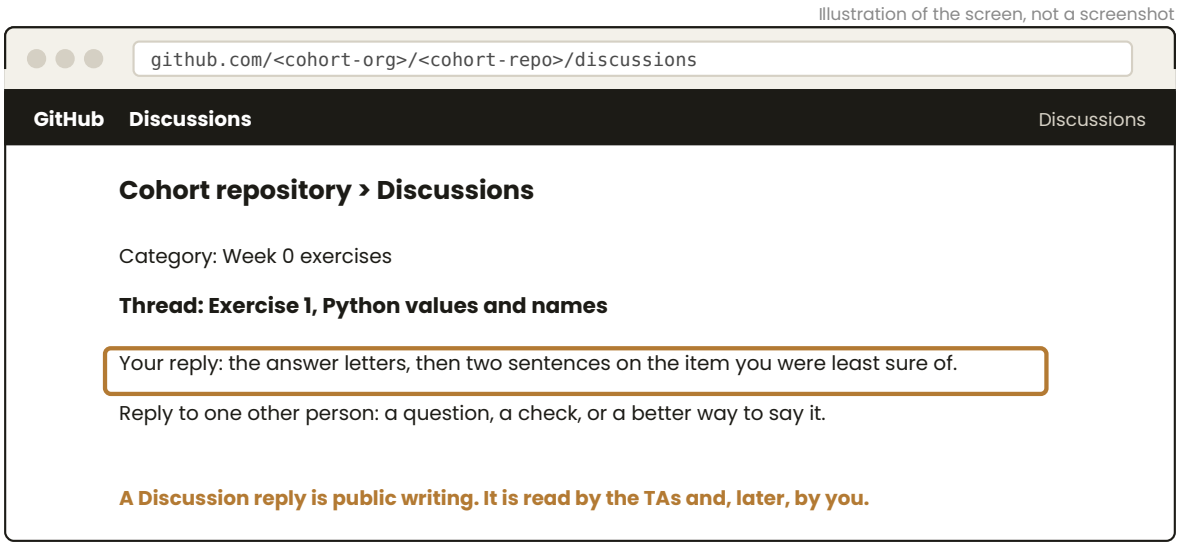


Figure 34. The cohort repository's Discussions tab, one thread per exercise. Your reply is the answer letters, then two sentences on the item you were least sure of, then one reply to someone else.

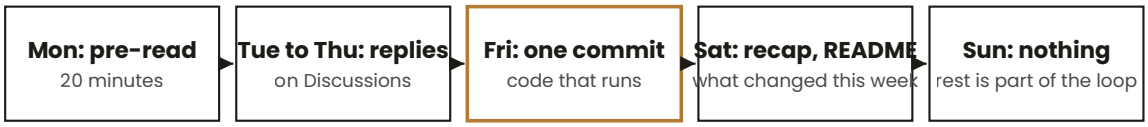
Exercises in this programme are answered as replies in the cohort repository's Discussions, in the thread for that exercise. The form is fixed so that TAs can read forty replies quickly: your answer letters on one line, two sentences on the item you were least sure of and why, and one reply to another learner that asks a question, checks a step or says the same thing better. Do not paste solutions to graded work; do paste tracebacks, because a traceback is how you ask for help at thirty minutes, the habit from Chapter 5.

The portfolio layout for twenty weeks

REPOSITORY	HOLDS	NAMED LIKE
`<username>`	The profile README	your username, exactly
`w00-diagnostic-<area>`	The six mini projects from this guide, one repository each, or one repository with six folders if you prefer fewer links	`w00-diagnostic-python`
`w03-build-1-data-to-insight`	Build 1, with the group's notebook, the deck and your own README	week, build number, topic
`w06-build-2-ml`, `w09-build-3-dl-llm`, `w12-build-4-retrieval`, `w15-build-5-agentic`	The later builds, same pattern	the same
`w17-capstone-<theme>`	The capstone, with the panel deck and the demo link	week, capstone, theme

The week prefix sorts the list into the story of the programme; a recruiter who scrolls it sees a progression without reading anything. Each repository carries the five-heading README. Pin your best four repositories on your profile; pins are the first thing shown.

The weekly loop



Twenty weeks of this loop is a portfolio with about a hundred commits and forty public answers. That is what an interviewer scrolls through.

Figure 35. Five moves a week, twenty weeks. The Friday commit is ringed because it is the one that builds the contribution graph an interviewer glances at.

The loop is small on purpose. Twenty minutes with the pre-read on Monday, your Discussion replies through the week, one real commit to a project repository on Friday, a README update on Saturday after the recap, and nothing on Sunday. Kept for twenty weeks, it produces about a hundred commits and forty public answers with no heroics.

Where this shows up in the work

The interview. The interviewer opens your profile on a phone. A README with three claims and three links gets read; a bare profile with default avatar gets closed.

The team's first week. A new hire who can create a branch, commit with a message and open a pull request on day one is productive on day one; the rest of Git can be learned on the job.

The lost phone. Recovery codes stored off the laptop are the difference between a ten-minute recovery and a portfolio that is gone.

Try this yourself

Checklist, verifiable by a TA in two minutes. Two-factor authentication is on and recovery codes are saved off the laptop; the profile README exists with three claims and three links; `w00-diagnostic-python` exists, is public, has the five-heading README and a notebook that runs top to bottom in a Codespace; at least one Discussion reply is posted in the required form; the best repositories are pinned.

Where this gets tested

Interview question. "Show me something you built." Tested: whether the anatomy exists. Strong answer opens a pinned repository whose README shows the result and whose notebook runs. Weak answer: "it is on my laptop".

Interview question. "What does a commit represent?" Tested: whether you understand version control at all. Strong answer: a saved, named, dated snapshot of the repository you can return to; commits are what a team reviews. Weak answer: "saving the file".

Interview question. "How do you set up your environment on a new machine?" Tested: reproducibility. Strong answer: open the repository in a Codespace, where the environment is defined in the repository, so nothing depends on the machine.

Glossary

TERM	PLAIN MEANING	WHERE IT APPEARED	EXAMPLE
Repository	A project folder with its full change history	Repositories section	`w00-diagnostic-python`
Commit	A named, dated snapshot of changes	Repositories section	"Add README skeleton"
README	The front page of a repository	Profile and repository sections	Five headings, Result included
Codespace	A cloud VS Code with the repository checked out	Codespaces section	Code, Codespaces, Create
Discussion	A public thread in a repository	Discussions section	One per exercise
Two-factor authentication	A second proof of identity at sign-in	Account section	Authenticator app plus recovery codes

Go deeper, in this order

STEP	RESOURCE	TIME	WHY THIS ONE
1	GitHub Docs, Creating an account on GitHub, docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github	10 min	The real screenshots for Figure 29
2	GitHub Docs, Managing your profile README, docs.github.com/en/account-and-profile/how-tos/profile-customization/managing-your-profile-readme	10 min	The real dialog for the profile repository
3	GitHub Docs, Quickstart for GitHub Codespaces, docs.github.com/en/codespaces/getting-started/quickstart	20 min	The canonical walkthrough for Figure 33
4	GitHub Skills, Code with GitHub Codespaces and Visual Studio Code, github.com/skills/code-with-codespaces	45 min	A guided, automated exercise you complete by committing
5	freeCodeCamp, "Git and GitHub for Beginners - Crash Course", youtube.com/watch?v=RGOj5yH7evk (May 2020)	70 min	Step by step on screen, from install to branches, using VS Code
6	freeCodeCamp, "How to Use Git and GitHub, Introduction for Beginners", freecodecamp.org/news/introduction-to-git-and-github	20 min	The same ground in text, with a profile README example

Chapter 9. The diagnostic, worked: every question in sequence, its answer, and why the other options fail

The forty questions follow here in the order they were set, Q1 to Q40, with the section breaks of the paper. Each carries the question as it was set, the four options with a verdict on each, the answer, and the reasoning step by step. Use the question map at the front of this guide to jump to any item. Reading time for the whole chapter: 44 minutes; about two minutes per item.

Section A. Python for data and GenAI, Q1 to Q12

Python for data and GenAI, Q1 to Q12. Chapter 1 carries the mechanisms.

Q1 of 40. Predict the output: types do not convert on their own

What does this code print?

```
orders = ["120", "80", "200"]
total = 0
for o in orders:
    total = total + o
print(total)
```

OPTION	TEXT	VERDICT
A	400, since the strings are converted to numbers first	Strings are never converted on their own; Python raises rather than guess.
B	12080200, since + joins the strings together	+ joins two strings, but total is an int, so the first + is int plus str, which has no meaning.
C	TypeError, since int and str cannot be added	**Correct.** total starts as the integer 0 and each o is a string, so the first + raises TypeError. Convert with int(o) before adding.
D	0, since the loop body never runs at all	The loop runs three times; the failure happens inside the first iteration.

Answer. C.

Step by step.

1. Read the types before the operator: total starts as the integer 0, and every o taken from orders is a string such as "120".
2. On the first pass the expression is 0 + "120", an int plus a str. Python's + has a meaning for two numbers and for two strings and no meaning across the two.

3. Python does not convert on its own, so it raises TypeError: unsupported operand type(s) for +: 'int' and 'str', and print(total) never runs.
4. The repair is one conversion, total = total + int(o), which makes the loop print 400.

Q2 of 40. Spot the bug, if there is one: two names, one list

Kavya expected q1 to stay as [4, 9, 7]. The code prints 4 9. Which statement is right?

```
q1 = [4, 9, 7]
q2 = q1
q2.append(3)
print(len(q1), sorted(q1)[-1])
```

OPTION	TEXT	VERDICT
A	There is a bug: q2 = q1 should have copied the list and Python failed to, so the change leaked into q1	Assignment is not a copy and never was; Python did what it always does.
B	The code is right: q2 = q1 makes both names refer to one list; q2 = q1.copy() would give a separate one	**Correct.** Assignment never copies a list. q1 and q2 are two names for one object, so an append through either name shows through both. q1.copy() or list(q1) makes a separate list.
C	append() cannot affect q1 because it was called on q2, so the printed 4 must come from somewhere else in the program	A method call acts on the object, not the name, so a change through q2 is visible through q1.
D	sorted() changed q1 in place, and that in-place sort is what added the fourth element to the list	sorted() returns a new list and leaves q1 untouched; append() added the element.

Answer. B.

Step by step.

1. q2 = q1 binds a second name to the same list object; nothing is copied.
2. q2.append(3) changes that one object, so q1 now refers to [4, 9, 7, 3] as well.
3. len(q1) is 4 and sorted(q1)[-1] is the largest value, 9, so the output 4 9 is exactly what the code says.
4. The expectation was wrong, the code was not; q2 = q1.copy() is what a separate list needs.

Q3 of 40. Trace the flow (pseudocode): accumulator and running best, strict comparison on a tie

This is pseudocode, not Python. What does it print?

```
SET running_total TO 0
SET best_day TO none
FOR EACH (day, revenue) IN [(Mon, 40), (Tue, 25), (Wed, 55), (Thu, 55)]
    running_total <- running_total + revenue
    IF best_day IS none OR revenue > best_revenue THEN
        best_day <- day
        best_revenue <- revenue
    END IF
END FOR
PRINT best_day, running_total
```

OPTION	TEXT	VERDICT
A	Thu 175	The tie goes to the earlier day because the comparison is strict; a later day would win only with >=.
B	Wed 120	The running total is never stopped; it adds every row, ending at 175.
C	Thu 55	The print shows the running total, not best_revenue.
D	Wed 175	**Correct.** The comparison is strict (>), so Thursday's 55 does not replace Wednesday's 55. The running total is 40 + 25 + 55 + 55 = 175.

Answer. D.

Step by step.

1. Set up two columns, running_total and best_day with best_revenue, and walk the four rows in order.
2. Mon 40: total 40, best Mon 40. Tue 25: total 65, best unchanged. Wed 55: total 120, 55 > 40 so best becomes Wed 55.
3. Thu 55: total 175; the test is revenue > best_revenue, 55 > 55, which is false, so best stays Wed.
4. The print shows best_day then running_total: Wed 175.

Q4 of 40. Predict the output: counting labels with dict.get

An LLM has labelled six support tickets. What does the code print?

```
labels = ["refund", "delivery", "refund", "refund", "delivery", "other"]
count = {}
for lab in labels:
    count[lab] = count.get(lab, 0) + 1
print(count["refund"], len(count))
```

OPTION	TEXT	VERDICT
A	3 3	**Correct.** count.get(lab, 0) returns 0 for a key it has not seen, so there is no KeyError. Three tickets are labelled refund and there are three distinct keys.
B	3 6	len(count) counts keys, and there are three distinct labels, not six.
C	6 3	The two values are swapped: refund count first, then the number of keys.
D	KeyError: 'refund'	.get with a default never raises KeyError; that is its purpose.

Answer. A.

Step by step.

1. count.get(lab, 0) returns the current count for lab, or 0 the first time a label is seen, so no KeyError is possible.
2. Walking the six labels gives refund 3, delivery 2, other 1.
3. count["refund"] is 3, and len(count) counts keys, not labels, so it is 3.
4. Output: 3 3.

Q5 of 40. Fix the code: reading a nested JSON response

The last line raises TypeError: list indices must be integers or slices, not str. Which replacement for the last line returns the text 'Refund approved'?

```
resp = {"choices": [{"message": {"role": "assistant", "content": "Refund approved"}}],
        "usage": {"prompt_tokens": 412, "completion_tokens": 9}}
text = resp["choices"]["message"]["content"]
```

OPTION	TEXT	VERDICT
A	`text = resp["choices"]["message"][0]["content"]`	The integer index belongs on choices, the list; message is a dict inside the first element.
B	`text = resp["choices"][0]["message"]["content"]`	**Correct.** resp['choices'] is a list, so it needs an integer index first; the message dict sits inside its first element.
C	`text = resp.choices.message.content`	A dict has no attributes named after its keys; dot access fails with AttributeError.
D	`text = resp["choices"]["message"]["content"][0]`	The list index is still missing at the choices level, and indexing the string at the end returns one character.

Answer. B.

Step by step.

1. Read the brackets: resp is a dict; resp["choices"] is a list (square brackets), and lists need an integer index.
2. The first element of that list is a dict with a "message" key; inside it, "content" holds the text.
3. The path therefore reads resp["choices"][0]["message"]["content"], one index per level, left to right.
4. Option B is that path; the others index the wrong level or treat the dict as an object.

Q6 of 40. Predict the output: truthiness drops zero as well as None

What does this code print?

```
sales = [1200, 0, 900, None, 300]
clean = [s for s in sales if s]
print(sum(clean) / len(clean), len(sales) - len(clean))
```

OPTION	TEXT	VERDICT
A	600.0 1	Only None removed would give 600.0 1; but 0 is also false in an if test and is removed too.
B	800 2	True division always returns a float, so the first number prints as 800.0, not 800.
C	TypeError, because None cannot be summed	The comprehension removes None before sum() ever sees it.
D	800.0 2	**Correct.** In an if test, 0 and None are both false, so the comprehension drops two values. sum(clean) / len(clean) is 2400 / 3, and / always gives a float: 800.0.

Answer. D.

Step by step.

1. The comprehension keeps a value only when "if s" is true; in Python both 0 and None are false.
2. So clean is [1200, 900, 300]: two values were dropped, and len(sales) - len(clean) is 2.
3. sum(clean) / len(clean) is 2400 / 3, and / always returns a float, so the first number prints as 800.0.
4. Output: 800.0 2.

Q7 of 40. Which line: integer division hides the fraction

Kavya's calculator gives 833.33 as the Plus average for the same rows, but the function returns 833. Which line causes the difference?

```
 1  def average_by_tier(rows):
 2      totals = {}
 3      counts = {}
 4      for tier, amount in rows:
 5          totals[tier] = totals.get(tier, 0) + amount
 6          counts[tier] = counts.get(tier, 0) + 1
 7      result = {}
 8      for tier in totals:
 9          result[tier] = totals[tier] // counts[tier]
10      return result
```

OPTION	TEXT	VERDICT
A	Line 9: the // operator discards the fraction	**Correct.** // is floor division and discards the fraction. totals[tier] / counts[tier] keeps it.
B	Line 5: totals should be written with += amount	totals[tier] = totals.get(tier, 0) + amount and += are the same operation; line 5 is correct.
C	Line 6: counts should start from 1, not 0	Starting counts at 1 would double-count the first row; line 6 is correct.
D	No line: the calculator is showing extra decimals	The calculator is right; the code discards the fraction with //.

Answer. A.

Step by step.

1. Lines 5 and 6 accumulate totals and counts correctly; += and the explicit form are identical.
2. Line 9 divides with //, floor division, which discards the fraction: 2500 // 3 is 833.
3. A calculator does true division and shows 833.33, so the difference comes from the operator, not the data.
4. Change // to / on line 9 and the function returns 833.333...

Q8 of 40. Predict the output: a default argument is created once

What does this code print?

```
def tag_order(order_id, tags=[]):
    tags.append(order_id)
    return tags

a = tag_order("K-101")
b = tag_order("K-102")
print(a, b)
```

OPTION	TEXT	VERDICT
A	['K-101'] ['K-102']	The default list is created once, not on every call, so the second call sees the first call's tag.
B	['K-101'] ['K-101', 'K-102']	return hands back the object itself, not a snapshot, so a and b are the same list and both show two tags.
C	['K-101', 'K-102'] ['K-101', 'K-102']	**Correct.** A default argument is created once, when the function is defined. Both calls append to that same list, so a and b are the same object. Use tags=None and create the list inside the function.
D	The output depends on the Python version	The behaviour is defined by the language and identical across Python versions.

Answer. C.

Step by step.

1. A default value is evaluated once, when def runs, so tags=[] creates one list object at definition time.
2. The first call appends "K-101" to that shared list and returns it; a is that list.
3. The second call, with no tags passed, appends "K-102" to the same list and returns it; b is the same object as a.
4. Both names show ['K-101', 'K-102']. The idiom tags=None with a fresh list inside the function gives one list per call.

Q9 of 40. Fix the code: defensive conversion: skip and count the bad rows

The comprehension crashes with ValueError on 'n/a'. Anand wants every bad value skipped and counted, whatever it looks like. Which rewrite does that?

```
def to_amount(text):
    return float(text.replace(",", ""))

rows = ["1,200", "950", "n/a", "300"]
amounts = [to_amount(r) for r in rows]
```

OPTION	TEXT	VERDICT
A	`amounts, rejected = [], []; for r in rows:; try:; amounts.append(to_amount(r)); except ValueError:; rejected.append(r)`	**Correct.** Only a try/except inside the loop catches every failing value and keeps a record of it. Skipping named strings misses the next bad value; a try around the whole loop drops every row after the first failure.
B	`amounts = []; for r in rows:; if r not in ("n/a", "NA", ""):; amounts.append(to_amount(r))`	It skips three named strings and crashes on the next bad value, and it counts nothing.
C	`amounts = []; try:; for r in rows:; amounts.append(to_amount(r)); except ValueError:; print("bad row found")`	One try around the whole loop stops at the first bad value and drops every row after it.
D	`amounts = [float(r.replace(",", "")) for r in rows if r.strip()]`	It removes the comma handling that '1,200' needs and still crashes on 'n/a'.

Answer. A.

Step by step.

1. The requirement has two parts: skip every bad value, whatever it looks like, and count what was skipped.
2. Only a try/except around the single conversion, inside the loop, meets both: each failure is caught where it happens and appended to rejected.
3. A filter on named strings (option B) handles today's bad values and crashes on tomorrow's; a try around the whole loop (option C) stops at the first failure and loses every later row; option D removes the comma handling and still crashes.
4. Option A is the only rewrite that finishes with amounts and rejected both filled.

Q10 of 40. Spot the bug: sorted() on tuples orders by the first element

The code should print the customer with the highest spend, but it prints ('Zara', 700). Why?

```
customers = [("Asha", 4200), ("Ravi", 9100), ("Zara", 700)]
top = sorted(customers)[-1]
print(top)
```

OPTION	TEXT	VERDICT
A	sorted() returns ascending order, so [-1] picks the smallest value rather than the largest	sorted() is ascending, so [-1] is the largest, which is correct; the problem is what it sorts by.
B	Tuples cannot be compared with each other, so the data must be a list of lists first	Tuples compare and sort fine; that is the mechanism producing the surprise.
C	700 is compared as text, and the string '7' sorts after '4' and '9'	700 is an integer and is never compared here; the names are compared first.
D	sorted() orders tuples by their first element, the name, unless a key is given	**Correct.** Tuples compare element by element from the first, so sorted() orders by name. sorted(customers, key=lambda c: c[1])[-1] gives the highest spend.

Answer. D.

Step by step.

1. sorted() compares tuples element by element, starting with the first element, which here is the name.
2. Alphabetically Asha, Ravi, Zara, so sorted(customers)[-1] is ("Zara", 700), the last name, not the highest spend.
3. To sort by spend, pass a key: sorted(customers, key=lambda c: c[1])[-1] returns ("Ravi", 9100).
4. Option D names the actual mechanism; the others describe things that are not happening.

Q11 of 40. Predict the output: braces inside a format template are placeholders

Kavya builds a classification prompt from a template. What happens on the second line?

```
template = ('Classify the ticket as one of {labels}. '
            'Reply as JSON: {"label": ...}. Ticket: {ticket}')
prompt = template.format(labels="refund, delivery, other", ticket="My parcel never
came")
```

OPTION	TEXT	VERDICT
A	The prompt is built with both placeholders filled and the JSON kept as written	format() cannot skip the JSON braces; it treats them as a placeholder and fails.
B	KeyError, since the braces around "label" read as a third placeholder	**Correct.** str.format() treats every {...} as a placeholder, so {"label": ...} is read as a field named "label" and raises KeyError. Double the braces ({{ }}) or build the JSON part separately.
C	SyntaxError, since double quotes cannot sit inside a single-quoted string	Double quotes inside a single-quoted string are valid Python; the string itself is fine.
D	The prompt is built but {ticket} is left unfilled because it comes last	format() is all or nothing; it raises before producing any output.

Answer. B.

Step by step.

1. str.format() scans the template for every pair of braces and treats each as a placeholder to fill.
2. The JSON example contains {"label": ...}, so format() looks for a keyword argument named "label": ... and finds none.
3. It raises KeyError: '"label"' on the second line; the prompt is never built, in whole or in part.
4. Escape literal braces by doubling them, {{ and }}, or assemble the JSON example outside the template.

Q12 of 40. Fix the code: a repeated inner list is one object

grid should be two independent rows of three zeros. After grid[0][1] = 7 it prints [[0, 7, 0], [0, 7, 0]]. Which first line gives [[0, 7, 0], [0, 0, 0]]?

```
grid = [[0] * 3] * 2
grid[0][1] = 7
print(grid)
```

OPTION	TEXT	VERDICT
A	`grid = [[0, 0, 0]] * 2`	[[0, 0, 0]] * 2 repeats one inner list exactly as the original did.
B	`grid = list([[0] * 3] * 2)`	list() copies the outer list; both entries still reference one inner list.
C	`grid = [[0] * 3 for _ in range(2)]`	**Correct.** [[0] * 3] * 2 repeats one inner list twice, so both rows are the same object; list() and copy() copy only the outer list. A comprehension builds a fresh inner list for each row.
D	`grid = [[0] * 3] * 2 and then grid = grid.copy()`	copy() is shallow: the outer list is new, the inner list is shared.

Answer. C.

Step by step.

1. [[0] * 3] * 2 builds one inner list and repeats the reference to it twice, so both rows are the same object.
2. Writing grid[0][1] = 7 changes that one object, which is why row 1 shows the 7 too.
3. list() and copy() copy only the outer list; the two inner references still point at one object.
4. A comprehension, [[0] * 3 for _ in range(2)], runs [0] * 3 twice and builds two independent rows.

Section B. SQL for data and GenAI, Q13 to Q20

SQL for data and GenAI, Q13 to Q20. The pipeline figure in Chapter 2 decides most of these.

Q13 of 40. Predict the result: cOUNT(*) against COUNT(column) with NULL

The orders table below is used for Q13 to Q16 and Q18. What does this query return?

ORDER_ID	CUSTOMER_ID	TIER	ORDER_DATE	AMOUNT	STATUS
1	C1	Plus	2026-04-02	1200	paid
2	C2	Basic	2026-04-03	NULL	paid
3	C1	Plus	2026-04-05	800	cancelled
4	C3	Student	2026-04-06	300	paid
5	C2	Basic	2026-04-09	500	paid

```
SELECT COUNT(*), COUNT(amount), SUM(amount)
FROM orders
WHERE status <> 'cancelled';
```

OPTION	TEXT	VERDICT
A	4, 4, 2000	COUNT(amount) skips NULL, so it returns 3, not 4.
B	4, 3, 2000	**Correct.** COUNT(*) counts rows, COUNT(amount) counts non-NULL amounts, and SUM ignores NULL. After the WHERE removes the cancelled order, four rows remain, three with an amount, summing to 2000.
C	5, 4, 2800	The WHERE clause removed the cancelled order; five rows are never counted.
D	4, 3, NULL	SUM ignores NULL rather than becoming NULL; it returns 2000.

Answer. B.

Step by step.

1. Apply WHERE first: status <> 'cancelled' removes order 3, leaving four rows.
2. COUNT(*) counts rows: 4. COUNT(amount) counts non-NULL amounts: order 2 is NULL, so 3.
3. SUM(amount) ignores NULL: 1200 + 300 + 500 = 2000.
4. Result: 4, 3, 2000.

Q14 of 40. Fix the query: integer division in SQL

On Postgres this query returns 0 for the orders table, where one order in five is cancelled. Which version returns 0.2?

```
SELECT SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) / COUNT(*) AS cancel_rate
FROM orders;
```

OPTION	TEXT	VERDICT
A	`SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) * 1.0 / COUNT(*)`	**Correct.** Both operands are integers, so Postgres does integer division and 1 / 5 becomes 0. Multiplying by 1.0 (or casting) makes the division decimal.
B	`ROUND(SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) / COUNT(*), 2)`	Rounding 0 to two decimals is still 0; the truncation happened before ROUND.
C	`COUNT(status = 'cancelled') / COUNT(*)`	COUNT(expression) counts rows where the expression is not NULL, which is every row, giving 5 / 5 = 1.
D	`AVG(status = 'cancelled')`	PostgreSQL has no AVG over booleans; the query errors.

Answer. A.

Step by step.

1. SUM(CASE ...) returns an integer and COUNT(*) returns an integer, so PostgreSQL performs integer division: 1 / 5 is 0.
2. Making either operand decimal changes the division: multiplying by 1.0 does it without changing the value.
3. ROUND of 0 is still 0; COUNT(expression) counts non-NULL rows, which is all five; AVG over a boolean is not defined on PostgreSQL.
4. Option A returns 0.2.

Q15 of 40. Predict the result: wHERE before grouping, HAVING after

Against the orders table, what does this query return?

```
SELECT tier, COUNT(*) AS n
FROM orders
WHERE status = 'paid'
GROUP BY tier
HAVING COUNT(*) > 1;
```

OPTION	TEXT	VERDICT
A	Plus 2 and Basic 2	Plus has two orders only if the cancelled one is counted, and WHERE removed it.
B	Basic 2, Plus 1 and Student 1	This is the result without HAVING; HAVING drops the groups with a single row.
C	No rows, because HAVING must refer to the alias n	HAVING may repeat the aggregate expression; the alias is not required.
D	Basic 2 only	**Correct.** WHERE removes the cancelled order before grouping, so Plus has one paid order. HAVING then keeps only groups with more than one row: Basic with 2.

Answer. D.

Step by step.

1. WHERE runs first and keeps paid orders only: Plus has 1 (order 3 was cancelled), Basic has 2, Student has 1.
2. GROUP BY tier forms three groups with those counts.
3. HAVING COUNT(*) > 1 keeps only groups with more than one row, which is Basic alone.
4. Result: Basic 2. HAVING may repeat the aggregate; it does not need the alias.

Q16 of 40. Spot the bug: nULL in a comparison

Kavya expects 4 (five orders minus the one at 800) but gets 3. Why?

```
SELECT COUNT(*) FROM orders WHERE amount <> 800;
```

OPTION	TEXT	VERDICT
A	<> is not valid in Postgres, so != must be used for the comparison to run	<> and != are both valid on PostgreSQL.
B	COUNT(*) skips any row that has a NULL in any of its columns	COUNT(*) counts every row it receives; the row was removed by WHERE, not by COUNT.
C	NULL compared with 800 is neither true nor false, so that row is dropped	**Correct.** A comparison with NULL yields unknown, and WHERE keeps only rows that are true, so the NULL-amount order is dropped along with the 800. Use amount <> 800 OR amount IS NULL if NULL rows should count.
D	The cancelled row is left out automatically because its status is not paid	Nothing in the query filters on status.

Answer. C.

Step by step.

1. Four orders have an amount, one has NULL. For the NULL row, amount <> 800 is neither true nor false but unknown.
2. WHERE keeps only rows whose condition is true, so the NULL row is dropped along with the row that equals 800.
3. Three rows remain, not the four Kavya expected.
4. To keep NULL rows, say so: WHERE amount <> 800 OR amount IS NULL.

Q17 of 40. Spot the bug and fix it: a WHERE on the right table turns a LEFT JOIN into an inner join

Anand wants one row per customer with the count of orders placed in 2026, including customers with none. The tables and the query are below. The query returns only C1 2 and C3 1. Which one-line change returns C1 2, C2 0, C3 1, C4 0?

customers

CUSTOMER_ID
C1
C2
C3
C4

orders

ORDER_ID	CUSTOMER_ID	ORDER_DATE
1	C1	2026-01-15
2	C2	2025-11-03
3	C1	2026-03-02
4	C3	2026-02-20

```
SELECT c.customer_id, COUNT(o.order_id) AS orders_2026
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_date >= '2026-01-01'
GROUP BY c.customer_id;
```

OPTION	TEXT	VERDICT
A	Move the date condition from WHERE into the ON clause	**Correct.** The LEFT JOIN keeps C2 and C4 with NULL order columns, but the WHERE on o.order_date then throws those rows away. A condition in ON filters the right side before the join, so every customer survives.
B	Change COUNT(o.order_id) to COUNT(*) so that the empty rows are counted	COUNT(*) would count the NULL-filled rows as 1, and the rows are already gone by the time COUNT runs.
C	Change LEFT JOIN to RIGHT JOIN so that the customers side is kept	RIGHT JOIN keeps every order instead of every customer; C4 would still vanish.
D	Add HAVING COUNT(o.order_id) >= 0 after GROUP BY	HAVING cannot restore rows WHERE has already removed.

Answer. A.

Step by step.

1. The LEFT JOIN keeps every customer; C2 and C4 get NULL in every order column because C2's only order is from 2025 and C4 has none.
2. WHERE o.order_date >= '2026-01-01' is then evaluated on those NULLs, is unknown, and the rows are dropped; the query behaves like an inner join.
3. Moving the date condition into the ON clause filters the orders side before the join, so every customer survives and C2 and C4 count 0.
4. Option A is that one-line change; the other options alter counting or join direction without addressing the NULL rows.

Q18 of 40. Predict the result: aggregates cannot sit in WHERE

Against the orders table, what happens when this runs on Postgres?

```
SELECT customer_id, SUM(amount) AS total
FROM orders
WHERE SUM(amount) > 1000
GROUP BY customer_id;
```

OPTION	TEXT	VERDICT
A	It returns the customers whose total is above 1000, in the order they appear	WHERE cannot evaluate SUM; the query does not run.
B	It fails: an aggregate is not allowed in WHERE; the condition belongs in HAVING	**Correct.** WHERE runs row by row before any grouping, so no aggregate exists yet and Postgres refuses the query. HAVING SUM(amount) > 1000 after GROUP BY is the right place.
C	It returns every customer, since WHERE runs before SUM exists and is quietly skipped	SQL never silently skips a condition; it raises an error.
D	It returns no rows, since SUM is NULL at the moment WHERE is evaluated	There is no NULL evaluation; the query is rejected before any row is read.

Answer. B.

Step by step.

1. WHERE runs row by row before any grouping, so at that point no aggregate exists to compare.
2. PostgreSQL rejects the query with the error that aggregate functions are not allowed in WHERE.
3. The condition belongs in HAVING, which runs after GROUP BY: HAVING SUM(amount) > 1000.
4. Option B is what happens; the options that return rows describe behaviour SQL does not have.

Q19 of 40. Predict the result: latest row per group with ROW_NUMBER

Farhan's team logs every LLM call made on a support ticket. The llm_calls table below is used for Q19 and Q20. What does this query return?

CALL_ID	TICKET_ID	MODEL	PROMPT_TOKENS	COMPLETION_TOKENS	COST
1	T1	small	400	100	0.02
2	T1	large	1200	300	0.30
3	T2	small	800	50	0.03
4	T3	large	600	200	0.30
5	T2	small	200	20	0.01

```
SELECT ticket_id, call_id
FROM (
  SELECT ticket_id, call_id,
         ROW_NUMBER() OVER (PARTITION BY ticket_id ORDER BY call_id DESC) AS rn
  FROM llm_calls
) t
WHERE rn = 1
ORDER BY ticket_id;
```

OPTION	TEXT	VERDICT
A	(T1, 1), (T2, 3), (T3, 4)	ORDER BY call_id DESC numbers the highest call first, so rn 1 is the latest call, not the earliest.
B	Five rows, because ROW_NUMBER never removes rows	The outer WHERE rn = 1 removes the other rows; five rows is the inner query's output.
C	An error, because a window function cannot be used to filter rows	A window function cannot sit directly in WHERE, and this one does not; it is computed in the inner query and filtered outside.
D	(T1, 2), (T2, 5), (T3, 4)	**Correct.** ROW_NUMBER() numbers each ticket's calls from the highest call_id down, and rn = 1 in the outer query keeps the latest call per ticket. The window function sits in a subquery, which is why filtering on it works.

Answer. D.

Step by step.

1. The inner query numbers each ticket's calls from the highest call_id downward: T1 gets call 2 as rn 1, T2 gets call 5 as rn 1, T3 gets call 4 as rn 1.
2. ROW_NUMBER never removes rows, so the inner query still has five rows.
3. The outer WHERE rn = 1 keeps one row per ticket; it works because the window value was computed in the inner query.
4. Result: (T1, 2), (T2, 5), (T3, 4).

Q20 of 40. Fix the query: join fan-out inflates a SUM

tickets holds one row per ticket with its category: T1 refund, T2 delivery, T3 refund. ticket_tags holds (T1, urgent), (T1, vip), (T2, urgent), (T3, vip). Anand asks for LLM spend by ticket category. The query returns refund 0.94 and delivery 0.04, which adds to 0.98, while the cost column of llm_calls sums to 0.66. Which change gives the right totals?

```
SELECT t.category, SUM(c.cost) AS spend
FROM llm_calls c
JOIN tickets t ON t.ticket_id = c.ticket_id
JOIN ticket_tags g ON g.ticket_id = t.ticket_id
GROUP BY t.category;
```

OPTION	TEXT	VERDICT
A	Change SUM(c.cost) to SUM(DISTINCT c.cost), so each cost value is added only once	SUM(DISTINCT) drops the second of the two genuine 0.30 costs and gives refund 0.32, wrong in a new way.
B	Add DISTINCT after SELECT, so repeated rows are removed before summing	SELECT DISTINCT removes duplicate output rows after aggregation; the SUM has already counted the repeats.
C	Drop the ticket_tags join; the question never uses tags and it repeats T1's calls	**Correct.** A ticket can carry several tags, so joining ticket_tags repeats every llm_calls row once per tag; T1's two tags double its spend. Drop the join. SUM(DISTINCT) would also merge the two genuine 0.30 costs into one.
D	Change JOIN tickets to LEFT JOIN tickets, so no ticket is lost from the totals	No ticket is lost; rows are duplicated, and LEFT JOIN changes nothing about that.

Answer. C.

Step by step.

1. The raw cost column sums to 0.66; the query returns 0.98, so 0.32 was added by the query itself.
2. T1 has two tags, so each of T1's two calls appears twice after the ticket_tags join; the extra 0.32 is exactly T1's calls counted once more.
3. The question never uses tags, so the join to ticket_tags is the defect; remove it and the totals are refund 0.62 and delivery 0.04.
4. SUM(DISTINCT) would also merge the two genuine 0.30 costs; SELECT DISTINCT acts after the SUM; LEFT JOIN changes nothing about duplication.

Section C. Numbers and reasoning, Q21 to Q28

Numbers and reasoning, Q21 to Q28. Every item is a rung on Chapter 3's ladder.

Q21 of 40. Reason with numbers: ratios of percentage changes

Kalpa Retail revenue fell 12 percent from Q1 to Q2 while the order count rose 5 percent. Which statement about the average order value must be true?

OPTION	TEXT	VERDICT
A	It fell by about 7 percent	Percentage changes divide; subtracting 5 from 12 is the common wrong move.
B	It fell by about 12 percent	It ignores the 5 percent rise in orders, which spreads the smaller revenue over more orders.
C	It cannot be determined without the number of customers	Average order value is revenue over orders; customers do not enter the definition.
D	It fell by about 16 percent	**Correct.** Average order value is revenue divided by orders: 0.88 / 1.05 is about 0.84, a fall of roughly 16 percent. Percentage changes divide; they do not subtract.

Answer. D.

Step by step.

1. Average order value is revenue divided by orders, so its change is the ratio of the two changes.
2. Revenue index 0.88, orders index 1.05: 0.88 / 1.05 is about 0.84.
3. That is a fall of about 16 percent; subtracting the percentages gives 7, which is wrong.
4. The customer count is not needed, since the definition involves only revenue and orders.

Q22 of 40. Reason with numbers: median against mean under one extreme value

Five Plus customers spent Rs 300, 350, 400, 450 and 9,000 this quarter. Meera wants one number on her slide for 'what a typical Plus customer spends'. Which number, and why?

OPTION	TEXT	VERDICT
A	400, the median, since one bulk buyer drags the mean above the other four	**Correct.** Four of the five customers spent 300 to 450; the mean of 2,100 describes none of them. The median, 400, is what 'typical' means here.
B	2,100, the mean, since it is the only number that uses every value	The mean is dragged to 2,100 by one buyer and describes none of the five.
C	300 to 9,000, the range, since any single number would mislead the board	A range refuses the choice the slide needs.
D	1,250, the midpoint between the mean and the median, as a compromise	The midpoint of the mean and the median is not a statistic; it has no meaning.

Answer. A.

Step by step.

1. Sort the values: 300, 350, 400, 450, 9,000. Four of five sit in a narrow band; one is far away.
2. The mean, 2,100, is pulled toward the outlier and describes nobody.
3. The median, 400, is the middle value and describes the typical customer, which is what the slide asks for.
4. The range avoids the decision; the midpoint of mean and median is not a statistic anyone uses.

Q23 of 40. Reason with numbers: compare rates, not counts

A monsoon discount email went to 20,000 customers and produced 400 orders. A plain email went to 5,000 customers and produced 150 orders. Which group responded better?

OPTION	TEXT	VERDICT
A	The discount group, since 400 orders is more than double 150	Counts from groups of different sizes cannot be compared; 400 of 20,000 is the lower rate.
B	The plain-email group, since 3 percent beats 2 percent	**Correct.** Compare rates, not counts: 150 / 5,000 is 3 percent and 400 / 20,000 is 2 percent. The plain email did better per person reached.
C	They are level once the group sizes are allowed for	Allowing for size is exactly what shows they are not level: 2 percent against 3 percent.
D	It cannot be judged without the revenue per order	Revenue per order answers a different question from response.

Answer. B.

Step by step.

1. Turn counts into rates: discount group 400 / 20,000 = 2 percent; plain group 150 / 5,000 = 3 percent.
2. The plain email produced more orders per person reached, so it responded better.
3. Four hundred beats 150 only as a count, and the groups differ four to one in size.
4. Revenue per order answers a different question; response rate is what was asked.

Q24 of 40. Two statements: percentage changes do not add

Statement I: A number that rises 10 percent and then falls 10 percent is back where it started.
Statement II: A number that falls 50 percent needs a rise of 100 percent to recover.

OPTION	TEXT	VERDICT
A	Only I is true	Statement I is false: 1.10 times 0.90 is 0.99.
B	Both are true	Statement I is false, so both cannot be true.
C	Only II is true	**Correct.** 1.10 x 0.90 = 0.99, so Statement I is false. Halving needs a doubling to recover, so Statement II is true.
D	Neither is true	Statement II is true: halving needs a doubling to recover.

Answer. C.

Step by step.

1. Statement I: 1.10 times 0.90 is 0.99, so a 10 percent rise then fall lands 1 percent below the start; false.
2. Statement II: after a 50 percent fall the value is 0.5 of the start; returning needs a factor of 2, which is a 100 percent rise; true.
3. Only II is true.

Q25 of 40. Reason with numbers: small counts move for no reason

A clinic with 50 bookings a week saw its no-show rate go from 10 percent to 14 percent this week. What is the fairest reading?

OPTION	TEXT	VERDICT
A	Two extra no-shows out of 50 sits inside normal week-to-week noise; watch a few more weeks	**Correct.** The rate moved from 5 no-shows to 7 out of 50. A change of two events sits well inside week-to-week noise for a count that small; several more weeks are needed before it means anything.
B	A rise from 10 to 14 percent is a 40 percent jump in the rate, which needs action this week	The 40 percent is a relative change on two events out of fifty; it is arithmetic on noise.
C	The rate has been miscalculated, since 7 out of 50 is 12 percent and not 14	7 out of 50 is 14 percent; the arithmetic in the option is wrong.
D	Nothing can be said either way until a proper significance test has been run	The size of the count already says this is within noise; no test is needed to say so.

Answer. A.

Step by step.

1. Convert the percentages to events: 10 percent of 50 is 5 no-shows; 14 percent of 50 is 7.
2. A change of two events on a base of fifty sits inside ordinary week-to-week variation for a count that small.
3. The relative jump of 40 percent is arithmetic on that noise; the arithmetic in option C is simply wrong; refusing to say anything until a test runs ignores what the numbers already show.
4. The fair reading is to watch several more weeks before acting.

Q26 of 40. Reason with numbers: base rates: precision from recall and prevalence

A fraud model flags 5 percent of all transactions. It catches 90 percent of the transactions that are actually fraudulent, and fraud is 1 percent of all transactions. Of the flagged transactions, roughly what share are actually fraud?

OPTION	TEXT	VERDICT
A	About 90 percent	90 percent is recall, the share of fraud caught; it says nothing about the share of flags that are fraud.
B	About 5 percent	5 percent is the flag rate, an input, not the precision.
C	About 1 percent	1 percent is the base rate, an input, not the precision.
D	About 18 percent	**Correct.** Of 1,000 transactions, 10 are fraud and the model catches 9 of them, while it flags 50 in total. So 9 of 50 flagged transactions, about 18 percent, are fraud. Catching 90 percent of fraud (recall) is not the same as 90 percent of flags being fraud (precision).

Answer. D.

Step by step.

1. Take 1,000 transactions. Fraud is 1 percent: 10 cases. The model catches 90 percent: 9 flagged fraud cases.
2. The model flags 5 percent of everything: 50 flags in total, so 41 of the flags are not fraud.
3. Precision is 9 / 50, about 18 percent: of the flagged transactions, roughly one in five is fraud.
4. 90 percent is recall, the share of fraud caught, and the other numbers are inputs, not the answer.

Q27 of 40. Reason with numbers: mix shift can reverse a total

Overall conversion fell from 5 percent to 4 percent, yet conversion rose in every single city. How can both be true?

OPTION	TEXT	VERDICT
A	They cannot both be true, since the total is just the cities added up; one figure must be wrong	Both can be true; the total is a weighted average, and the weights changed.
B	The mix shifted toward cities with lower conversion, so the total fell as each city rose	**Correct.** The overall rate is a weighted average of the city rates. If more customers now come from low-conversion cities, the total can fall while every city improves. This is a mix shift.
C	Rounding in the city figures hides small falls that only show up in the total	Rounding cannot reverse the direction of every city at once.
D	Seasonality moves the total without moving any single city's own figure	The total is made of the cities; nothing can move it without moving them.

Answer. B.

Step by step.

1. The overall rate is a weighted average of the city rates, weighted by how many customers came from each city.
2. If the share of customers from low-converting cities grew, the total can fall while every city's own rate rises.
3. Rounding cannot reverse a direction in every city, and seasonality that moved the total would show in the cities too.
4. The mix shifted; the question to ask is where the traffic went.

Q28 of 40. Estimate: token economics, multi-step estimate

Farhan plans one LLM call per support ticket. Suppose the model bills Rs 40 per million input tokens and Rs 160 per million output tokens, and a call uses about 1,200 input tokens and 200 output tokens. At 1,000 tickets a day for 30 days, the monthly bill is closest to

OPTION	TEXT	VERDICT
A	Rs 1,440	Rs 1,440 is the input tokens alone; output tokens cost Rs 960 more.
B	Rs 6,720	Rs 6,720 applies the Rs 160 output price to all 42 million tokens.
C	Rs 2,400	**Correct.** Input: 1,000 x 1,200 x 30 = 36 million tokens, Rs 1,440 at Rs 40 per million. Output: 6 million tokens, Rs 960 at Rs 160 per million. Total Rs 2,400 a month.
D	Rs 24,000	Rs 24,000 is a decimal slip: the token counts are in millions and the prices are per million.

Answer. C.

Step by step.

1. Input tokens: 1,000 tickets x 1,200 tokens x 30 days = 36 million; at Rs 40 per million that is Rs 1,440.
2. Output tokens: 1,000 x 200 x 30 = 6 million; at Rs 160 per million that is Rs 960.
3. Total Rs 2,400 a month.
4. Rs 1,440 forgets the output; Rs 6,720 applies the output price to every token; Rs 24,000 slips a decimal.

Section D. Case and scenarios, Q29 to Q34

Case and scenarios, Q29 to Q34. The case table is repeated once below; Q33 and Q34 are the language-model scenarios from Chapter 4.

Q29 of 40. Case: Kalpa Retail, Q1 against Q2: which branch moved

Meera Raghavan, CEO of Kalpa Retail, has one question for the analytics team: revenue fell from Q1 to Q2, where did it go? Kavya Nair pulled the table below. Use it for Q29 to Q32. Revenue is in Rs lakh.

TIER	Q1 CUSTOMERS	Q1 ORDERS	Q1 REVENUE	Q2 CUSTOMERS	Q2 ORDERS	Q2 REVENUE
Plus	10,000	30,000	360	10,000	27,000	324
Basic	40,000	60,000	420	44,000	64,000	400
Student	5,000	6,000	24	8,000	9,000	36
Total	55,000	96,000	804	62,000	100,000	760

Which line best explains most of the fall?

OPTION	TEXT	VERDICT
A	Plus customers ordered less often at the same order value; that tier carries most of the fall	**Correct.** Plus revenue fell 36 lakh (orders down 10 percent at an unchanged Rs 1,200 per order), Basic fell 20 lakh, Student rose 12 lakh. Plus alone explains most of the net 44 lakh fall.
B	Basic customers paid less per order than in Q1; that tier carries most of the fall	Basic's lower order value cost 20 lakh; Plus cost 36 and carries most of the fall.
C	Student growth pulled spending away from the Plus tier, which is why Plus orders fell by a tenth	The table shows no link between Student growth and Plus orders; it is a story, not a finding.
D	Fewer customers overall placed orders in Q2, which lowered revenue across every tier	Customers rose from 55,000 to 62,000; fewer customers is contradicted by the table.

Answer. A.

Step by step.

1. Read the deltas first: Plus 360 to 324 is a fall of 36 lakh; Basic 420 to 400 is 20; Student 24 to 36 is a rise of 12; net 44 down.
2. Plus carries most of the fall. Inside Plus, customers are flat at 10,000, orders fell 30,000 to 27,000, and revenue per order stayed at Rs 1,200.
3. So Plus customers ordered less often at the same order value.
4. Basic's lower order value is real but smaller; the customer count rose overall; nothing links Student growth to the Plus fall.

Q30 of 40. Case: Kalpa Retail, Q1 against Q2: test a hypothesis against the table

Kavya's first hypothesis is that Plus customers are downgrading to Basic. What does the table say about it?

OPTION	TEXT	VERDICT
A	It supports it, since Basic gained 4,000 customers	Basic gaining customers is consistent with new sign-ups; it does not show where they came from.
B	It supports it, since the Basic order value fell	Basic's order value says nothing about Plus customers moving.
C	It weakens it, since the Plus customer count did not fall	**Correct.** Plus customers were 10,000 in both quarters. If Plus customers had moved to Basic, the Plus count would have dropped. The table weakens the hypothesis.
D	It cannot be judged without customer-level churn data from both quarters	The table already carries the test: the Plus customer count did not fall.

Answer. C.

Step by step.

1. If Plus customers moved to Basic, the Plus customer count would fall unless upgrades replaced them exactly.
2. The table shows Plus at 10,000 in both quarters, so the count did not fall.
3. That weakens the hypothesis with evidence already on the page; the Basic rise is consistent with new customers, not with downgrades.
4. Churn data would refine it, but the table already gives a verdict.

Q31 of 40. Case: Kalpa Retail, Q1 against Q2: the fair comparison

A monsoon discount was given to some Basic customers in Q2. Meera asks whether it caused the rise in Basic orders. Which comparison is the fairest one available?

OPTION	TEXT	VERDICT
A	Basic Q2 orders against Basic Q1 orders, since the before-and-after is the change in question	Before and after mixes the discount with season, festivals and trading days.
B	Discounted Basic customers against non-discounted Basic customers, same weeks	**Correct.** The fair comparison holds everything else constant: same tier, same weeks, the discount as the only difference. Before-and-after and cross-tier comparisons mix the discount with every other change.
C	Basic Q2 orders against Plus Q2 orders, since Plus had no discount	Plus is a different tier with different behaviour; the comparison varies the wrong thing.
D	Basic Q2 revenue against the plan line, since the plan assumed no discount	A plan line is a target, not a comparison group.

Answer. B.

Step by step.

1. A fair comparison varies only the thing being tested and holds everything else still.
2. Basic Q2 against Basic Q1 varies the discount together with season, festivals and trading days; Basic against Plus varies the tier; revenue against plan compares with a target, not a group.
3. Discounted Basic customers against non-discounted Basic customers over the same weeks isolates the discount.
4. That is the fairest comparison the data allows.

Q32 of 40. Case: Kalpa Retail, Q1 against Q2: like-with-like windows before presenting

Before you put the Plus finding in front of Meera, which check protects the finding most?

OPTION	TEXT	VERDICT
A	Ask for the Plus tier's marketing budget for Q2 and compare it with Q1's	A budget explains a fall; it does not test whether the fall is real.
B	Re-add the six revenue figures in a spreadsheet to confirm that the totals match the table exactly	Re-adding the sums checks arithmetic, not the claim.
C	Compute the Plus tier's share of total revenue in each quarter and compare	The share of revenue re-describes the same numbers and protects nothing.
D	Check that Q1 and Q2 have the same trading days and that no festival sat in one quarter only	**Correct.** A quarter with fewer trading days, or a festival that sat in Q1 only, can create the whole fall on its own. Checking the window protects the finding; the other options describe or re-add the same numbers.

Answer. D.

Step by step.

1. A finding about a fall between two quarters rests on the quarters being comparable.
2. If Q1 and Q2 differ in trading days, or a festival sat in one only, the entire fall can be an artefact of the window.
3. Re-adding the sums checks arithmetic, not the claim; the budget explains rather than tests; the revenue share re-describes the same numbers.
4. Check the window first; it costs one calendar lookup.

Q33 of 40. Scenario: LLM intuition: sampling variance and an under-specified prompt

Farhan's team uses an LLM to label each support ticket as refund, delivery or other. The prompt says only 'Label this ticket'. Sending the same ticket twice gives refund once and delivery the second time. Which explanation and first fix fit best?

OPTION	TEXT	VERDICT
A	Sampling above zero temperature varies the reply, and the prompt never names the labels; lower it and list the three labels with an example each	**Correct.** Sampling with a temperature above zero makes replies vary, and a prompt that never lists the allowed labels leaves the model to invent them. Lower the temperature and give the label set with an example each. API calls carry no memory of each other unless the history is sent.
B	The model remembers the first call and changed its mind on the second; clear its conversation history before every call so each starts fresh	Calls carry no memory unless the history is sent; the model cannot change its mind between calls.
C	The ticket text must contain a typo or odd formatting that confuses the model; clean the data before calling again and re-check	The same text was sent twice; a typo would produce the same wrong answer twice, not two answers.
D	The model is too small for a three-way classification; switch to the largest model available and re-run the whole batch	A larger model still samples above zero temperature and still needs the label set.

Answer. A.

Step by step.

1. Two things in the setup can produce different labels: sampling above zero temperature, which picks from the scores rather than always the top one, and a prompt that never names the allowed labels, which leaves the model to invent the set.
2. A call carries no memory of an earlier call, so the model cannot have changed its mind; a typo does not explain two different answers to identical text; a larger model still samples.
3. The first fix is the prompt: list the three labels with an example each and ask for the label alone; the second is a lower temperature.
4. Then validate every reply against the allowed set.

Q34 of 40. Scenario: LLM intuition: a small hand-labelled set is the fastest trust check

Kavya's prompt asks a model to rate the sentiment of each customer review from 1 to 5. On 200 reviews the ratings look plausible. Which check tells her fastest whether the ratings can be trusted?

OPTION	TEXT	VERDICT
A	Ask the model to add a confidence score to each rating and keep the high ones	A confidence score is more model output; it cannot check the model.
B	Hand-rate a random 30 reviews herself and compare with the model's ratings	**Correct.** Ratings that look plausible are not evidence. Hand-rating a random sample and comparing gives a measured agreement rate within an hour; confidence scores and a second model only add more unverified output.
C	Run the prompt through a second model and average the two ratings for each review	Two models agreeing measures agreement, not accuracy.
D	Add 'be accurate and careful' to the prompt and re-run the whole batch	Changing the prompt changes the input; it measures nothing.

Answer. B.

Step by step.

1. Plausible output is not evidence; the question is how to produce a number that says how often the ratings are right.
2. Hand-rate a random sample of 30, compare with the model's ratings, and compute agreement, overall and per rating.
3. Confidence scores are more model output; a second model agreeing is two opinions, not a measurement; adding words to the prompt changes the input without measuring anything.
4. Thirty items take under an hour and give a rate you can defend.

Section E. Judgment calls, Q35 to Q40

Judgment calls, Q35 to Q40. Each situation has a best action and a worst action, and the reasoning names both.

Q35 of 40. Best and worst

You sent Anand Iyer, the Finance Controller, the quarterly revenue figure yesterday. This morning you find your query counted returned orders as sales, so the figure is about 4 percent too high, and it is already in a board deck. Mark the best action and the worst action.

OPTION	TEXT	VERDICT
A	Fix the query so next month's figure is right and say nothing about this one, since 4 percent sits inside normal variance	**Worst.** Saying nothing about a known error in a board figure is the one action that destroys trust when it surfaces later, and it always surfaces.
B	Ask a peer to re-run your query and send a correction only once they confirm the error is real	Asking a peer to re-run the query delays a correction you already have; the error is known.
C	Send Anand the corrected figure now, with the cause and size of the error	**Best.** Sending the correction yourself, now, with the cause and the size, is the only option that protects the board figure and your credibility at once.
D	Tell your reporting manager and let them decide whether Finance needs to hear about it	Passing the decision upward delays it and makes someone else responsible for your number.

Answer. Best C, worst A.

Step by step.

1. The figure is already in a board deck, so every hour of delay raises the cost of the correction.
2. Best: send Anand the corrected figure now, with the cause and the size, before anyone else finds it.
3. Worst: saying nothing because the error is small treats a known error in a board figure as acceptable; it will surface, and then the question is why you did not say.

Q36 of 40. Best and worst

During an ungraded exercise you got the answer from an AI assistant and submitted it without understanding it. The trainer asks you to explain your approach to the room. Mark the best action and the worst action.

OPTION	TEXT	VERDICT
A	Ask the trainer to come back to you once you have had time to study the answer properly	Asking for time is honest but avoids the question; the room learns nothing and neither do you.
B	Say you used an AI assistant, then walk through what you do and do not yet understand	**Best.** Naming the tool and what you do not yet understand is the honest answer and the one that gets you help on the spot.
C	Ask the person next to you, who solved it without help, to explain the approach instead	Handing the explanation to a neighbour avoids the question and shows the room what you did not do.
D	Present the answer as your own reasoning and improvise wherever the logic is unclear to you	**Worst.** Presenting borrowed reasoning as your own compounds the gap and is found out within minutes of the first question.

Answer. Best B, worst D.

Step by step.

1. The question in the room is about your understanding, and the only durable answer is an honest one.
2. Best: say you used an assistant, then walk through what you do and do not yet understand; that is how you get help on the spot.
3. Worst: presenting borrowed reasoning as your own is discovered by the first follow-up question and costs more than the gap it hides.

Q37 of 40. Best and worst

During your group's presentation a group-mate says, in front of the panel, that your chart is misleading. Mark the best action and the worst action.

OPTION	TEXT	VERDICT
A	Say nothing now, since the panel is watching, and fix the chart after the presentation	Staying silent leaves the panel with the doubt and the chart uncorrected.
B	Point out that their own section had errors too, so the panel hears both sides	**Worst.** Retaliating in front of the panel damages both of you and answers nothing about the chart.
C	Defend the chart, since you checked the numbers twice before the presentation	Defending before hearing the specific point closes the conversation that could fix the chart.
D	Ask what specifically misleads and offer to fix it before the presentation moves on	**Best.** Asking what specifically misleads, in front of the panel, turns criticism into a fix and shows the panel how you work.

Answer. Best D, worst B.

Step by step.

1. Criticism in front of a panel is uncomfortable and also useful, if you treat it as a fix.
2. Best: ask what specifically misleads and offer to fix it before the presentation moves on.
3. Worst: retaliating with your group-mate's errors damages both of you and answers nothing about the chart.

Q38 of 40. Best and worst

A take-home is due tonight. Your notebook runs, but the answer to the last question is wrong and you know why; the fix needs two more hours. Mark the best action and the worst action.

OPTION	TEXT	VERDICT
A	Submit on time with a note saying which part is wrong, why, and when the fix follows	**Best.** Submitting on time with a note that names the wrong part, the cause and the fix date keeps both the deadline and your credibility.
B	Message the TA that you will submit tomorrow with the fix included, since correctness matters more	Missing the deadline is acceptable when necessary, but here a note would have kept both the deadline and the truth.
C	Submit as it is and say nothing, since the notebook runs and the deadline is what counts	**Worst.** Knowingly submitting a wrong answer as right is exactly the failure the note would have prevented.
D	Submit only the parts that are right and quietly drop the last question from the notebook	Dropping the question hides the gap instead of naming it.

Answer. Best A, worst C.

Step by step.

1. Two things matter: the deadline and the truth about the last answer.
2. Best: submit on time with a note naming the wrong part, why it is wrong and when the fix follows; the reader loses nothing.
3. Worst: submitting silently because the notebook runs presents a known wrong answer as right.

Q39 of 40. Best and worst

You have spent 90 minutes stuck on an environment error while the session moves on without you. Mark the best action and the worst action.

OPTION	TEXT	VERDICT
A	Ask the person next to you to do the setup on your machine so you can catch up with the session	Someone else fixing your setup solves today and teaches nothing; you will meet the same error alone.
B	Wait until the end of the day and email a screenshot of the error to the TA	Waiting until the end of the day loses the whole session.
C	Post the exact error text in the Discussions thread and continue with the parts that need no setup	**Best.** Posting the exact error text is the fastest route to a fix, and working on the parts that need no environment keeps the day productive.
D	Keep trying alone, since asking now would show the room you cannot handle the basics	**Worst.** Ninety minutes alone is already too long; asking early is the habit the programme depends on.

Answer. Best C, worst D.

Step by step.

1. Ninety minutes alone on an environment error is already the mistake; the session has moved on.
2. Best: post the exact error text where the TAs will see it and keep working on the parts that do not need the environment.
3. Worst: continuing alone to avoid looking weak loses the day and teaches nothing.

Q40 of 40. Best and worst

Anand says your analysis 'cannot be right' because it contradicts his experience, and asks you to change the conclusion before the meeting. Mark the best action and the worst action.

OPTION	TEXT	VERDICT
A	Change the conclusion; he has run Finance for years and knows the business better than the data	**Worst.** Changing a conclusion because of pushback abandons the evidence and makes the analysis worthless to everyone, including him.
B	Walk him through the path from raw table to final number and ask which step he doubts	**Best.** Walking through the path from raw table to number lets him point at the step he doubts, which is either a real error or the end of the disagreement.
C	Tell him the numbers do not lie and keep the conclusion exactly as it is	Holding the finding is right; closing the conversation with 'the numbers do not lie' loses the chance to find a real error.
D	Escalate to Meera at once so she can decide between his experience and your analysis	Escalating skips the conversation that would settle it and turns a data question into a dispute.

Answer. Best B, worst A.

Step by step.

1. Pushback from an experienced stakeholder is information, not an instruction.
2. Best: walk him through the path from raw table to final number and ask which step he doubts; either a real error surfaces or the disagreement ends.
3. Worst: changing the conclusion because he pushed abandons the evidence and makes the analysis worthless to everyone, including him.

The path: what to watch, read and build, in order

Everything below was opened and checked on 29 September 2026. Times are what a working adult needs, not the video's length alone.

Videos to watch, to recap

ORDER	VIDEO	LENGTH	RECAPS
1	Ned Batchelder, "Facts and Myths about Python names and values", PyCon 2015, youtube.com/watch?v=_AEJHKGk9ns	30 min	Chapter 1, the board
2	3Blue1Brown, "Bayes theorem, the geometry of changing beliefs", youtube.com/watch?v=HZGCoVF3YvM	15 min	Chapter 3, base rates
3	3Blue1Brown, "But what is a GPT? Visual intro to transformers", youtube.com/watch?v=wjZofJX0v4M	27 min	Chapter 4, tokens and sampling
4	Andrej Karpathy, "[1hr Talk] Intro to Large Language Models", youtube.com/watch?v=zjkBMFhNj_g	60 min	Chapter 4, the whole picture
5	freeCodeCamp, "SQL Tutorial, Full Database Course for Beginners", youtube.com/watch?v=HXV3zeQKqGY, the joins and nested queries chapters first	4 h in total	Chapter 2
6	Corey Schafer, "Python Pandas Tutorial (Part 1)", youtube.com/watch?v=ZyhVh-qRZPA, then parts 4 and 8 of the playlist	25 min each	Chapter 6
7	freeCodeCamp, "Git and GitHub for Beginners, Crash Course", youtube.com/watch?v=RGOj5yH7evk	70 min	Chapter 8, with the step-by-step screen help this guide's illustrations point at

Reading, to go one level deeper

ORDER	READING	TIME	DEEPENS
1	nedbatchelder.com/text/names.html	20 min	Chapter 1
2	docs.python.org/3/tutorial/errors.html	25 min	Chapter 1, tracebacks
3	postgresql.org/docs/current/tutorial-sql.html	40 min	Chapter 2
4	learnsql.com/blog/window-functions	25 min	Chapter 2, windows
5	plato.stanford.edu/entries/paradox-simpson	30 min	Chapter 3, mix shift
6	developers.google.com/machine-learning/crash-course, the Classification module	45 min	Chapters 3 and 5
7	platform.claude.com/docs/en/build-with-claude/prompt-engineering and platform.openai.com/docs/guides/prompt-engineering	50 min	Chapter 4
8	pandas.pydata.org/docs/user_guide/10min.html	30 min	Chapter 6
9	support.microsoft.com/office/18fb0032-b01a-4c99-9a5f-7ab09edde05a	15 min	Chapter 7
10	The four GitHub Docs pages in Chapter 8	50 min	Chapter 8

The six mini projects, as a week

DAY	BUILD	REPOSITORY	PROVES
Wednesday	Mini project 1, Python: clean and total	`w00-diagnostic-python`	Types, the loop, `try` per row, the assert
Wednesday	Mini project 2, SQL: Meera's question in one statement	`w00-diagnostic-sql`	Pipeline, LEFT JOIN with ON, ROW_NUMBER
Thursday	Mini project 3, numbers: the tier decomposition	`w00-diagnostic-numbers`	Rates, mix shift, the tree
Thursday	Mini project 4, language models: a golden set	`w00-diagnostic-llm`	The four-part prompt, agreement per label
Saturday	Mini project 5, business: the one-paragraph answer	`w00-diagnostic-business`	Hypotheses against the table, the window check
Saturday	Mini project 6, pandas: project 1 in six lines, plus the Excel pivot	`w00-diagnostic-pandas`	The chain, validate on merge, the pivot that agrees

Each project ends when its self-check passes and its README shows the result. Post the repository link in the Week 0 thread on Discussions; the one-to-one this week starts from your diagnostic report and ends with whichever of these you have built.